Bouncer: Competence Auditing for Set-Local Learned Microarchitectural Controllers

Anonymous authors
Paper under double-blind review

Abstract

Learned microarchitectural controllers can outperform fixed heuristics on average yet regress under workload shift or adversarial steering. We present **Bouncer**, a runtime monitor and trust gate that measures *realized competence*: it secretly set-duels a learned controller against a vetted fallback on hidden, per-epoch-reshuffled sets and gates the controller when its measured advantage falls below a threshold.

Under bounded rewards, secret uniform sampling, and explicit assumptions on detection, re-trust, and false alarms, Bouncer bounds expected window-normalized regret over audited declared domains by three terms: fully-open detection and re-trust windows, audit exposure proportional to the secret dueling fraction and drop duration, and false-alarm switching. A per-domain result gives an expectation-level mimicry floor and, when per-window estimates are independent conditional on the strategy, an exponentially decreasing probability of sustained realized evasion. It gives no single-window guarantee and does not cover sub-resolution harm.

Given representative within-domain sampling, faithful auditing requires two controller-side properties. First, reward must be *set-local*, so a decision and its outcome can be attributed to the same dueling pool. Cache replacement satisfies this spatial condition; prefetching does not because a prefetch can benefit a different set. Second, the policy contrast must be *reseed-identifiable*, so reshuffling leaders does not erase the measured advantage. A stateless reward is sufficient but not necessary: a stateful reward with state-invariant contrast remains auditable. We argue that each condition is necessary and demonstrate both boundaries; a full sufficiency theorem remains future work.

In a released, seeded synthetic harness satisfying both conditions, the estimator tracks ground truth (R^2=0.997), the attacked steady-state floor remains within 0.64% of the fallback, detection takes two audit windows at the operating point, and the clean-workload tax is 0.35%. Across 50 seeds, Bouncer detects all mimicry attacks (50/50; Wilson 95% lower bound 0.93), whereas an input-distribution monitor detects none (0/50; upper bound 0.07). A leak ablation shows detection collapse between 0.6 and 0.8 leaked assignment fraction, while a slow-bleed attack exposes the finite-resolution limit.

ChampSim experiments serve as boundary studies rather than full-system validation: prefetch reward is not set-local, and secret reseeding confounds a path-dependent cache-replacement reward. An illustrative 16-bit representation of the released Tier-A dimensions requires about 404 bytes (0.15% of a 256 KB static random-access memory). Register-transfer-level area, energy, and timing remain unmeasured.

1 Introduction

Modern processors increasingly hand small *learned* models control of decisions that used to be made by fixed, hand-tuned rules. Consider the *cache*—a small, fast memory that holds recently or soon-to-be-used data so the processor rarely waits on slow main memory. Two decisions dominate a cache's performance: which data to *prefetch* (fetch early, before it is asked for) and which block to *evict* when the cache is full (*replacement*).

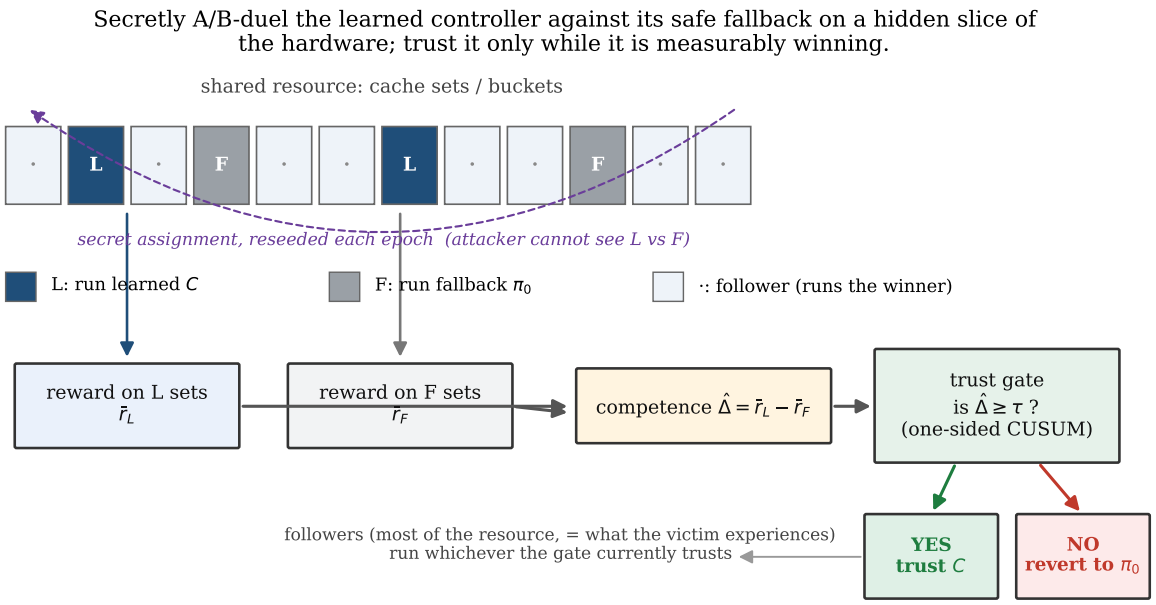


Figure 1: **Secretly A/B-duel the learned controller against its safe fallback on a hidden slice of the hardware; trust it only while it is measurably winning.** A few randomly chosen *sets* (address buckets in the cache) run the learned controller, a few run the fallback, and the assignment is a hardware secret reshuffled each epoch (a fixed interval). Because the winner is *measured*, expected regret is bounded under the stated assumptions. With representative within-domain sampling, an intact secret, and temporal independence, sustained audit evasion by a degrading strategy becomes exponentially unlikely; no single-window guarantee is claimed.

Learned versions of both now beat the classic heuristics—a reinforcement-learning prefetcher Bera et al. (2021), an imitation-learned replacement policy Jain & Lin (2016); Shi et al. (2019)—and similar learned controllers are arriving for memory scheduling and branch prediction Ipek et al. (2008); Jiménez & Lin (2001). The tempting move is to ship them and trust the average: they were validated to win, so let them run.

"Wins on average," though, is a property of the workloads a controller was validated on, and it does not travel. On a new program or a new execution phase a learned controller can quietly become *worse* than the simple heuristic it displaced—a silent regression that aggregate performance counters never flag. Worse still, a shared on-chip controller is exercised by *every* program on the core, so a hostile co-running program can *steer* it: craft memory accesses that push the controller into its bad regime while keeping the aggregate counters looking normal Guo et al. (2022); Evtyushkin et al. (2018). Nothing in a deployed system checks, at runtime, whether the learned controller is still earning its keep. The obvious check—watch the controller's *inputs* for distribution shift—is both insufficient and unsafe: an attacker can hold the input statistics in-distribution while wrecking the controller's behavior (a *mimicry* attack), and an input monitor is blind to that by construction.

We instead check the thing that actually matters: *realized competence*, whether the learned controller is beating the fallback *right now*—and we show this can be done cheaply, provably, and without trusting the inputs. The mechanism (Figure 1) builds on the fact that a cache slices its storage into many *sets* (address buckets that each memory address maps into). It repurposes *set-dueling*—running two candidate policies on different sets and comparing them—a technique already in commercial caches: we secretly dedicate a randomly chosen handful of sets to run the learned controller and another handful to run the safe fallback, compare the reward they earn, and let the rest of the cache follow the current winner. If the learned controller stops winning, the gate reverts to the fallback. Measurement yields an expected-regret bound. Secrecy

supplies a different property: under the per-domain sampling and temporal assumptions of Proposition 1, a degrading strategy is exponentially unlikely to evade the audit for many consecutive windows. A single realized window has no such guarantee.

The central finding is *where this works and where it does not.* Secret set-dueling can attribute reward to the right policy only if that reward is *set-local*: the outcome of a decision made on a set is realized on that same set. Cache *replacement* is set-local—an eviction on a set is scored by the hits and misses on that set—and is the clean case. *Prefetching is the instructive boundary*: a prefetch triggered on one set benefits a *different* set, so no per-set reward captures it, and we confirm on a real simulator that not even the controller's own reward rescues the audit (Section 11.14). A second condition surfaces in the real-trace simulator study, and it is *not* statelessness. The secret reshuffle erases any long-run, policy-specific state a reward accumulates, so what the audit needs is that the policy *contrast*—the C-minus-fallback advantage—survive that erasure: the reward must be *reseed-identifiable.* A (near-)stateless reward is the clean way to get this, but not the only way: we show an additive-warmup reward that is *stateful* yet stays auditable because its contrast is state-invariant, and a multiplicative-warmup reward (the cache case) that fails because its contrast scales with the state the reshuffle destroys. Together, set-locality and reseed-identifiability are the two conditions we identify as *governing* which controllers this kind of audit can certify: we argue each is necessary and demonstrate the boundary on real and synthetic controllers, leaving a complete sufficiency theorem to future work—the paper's organizing result, with prefetching and one named security limitation as its honest boundaries.

Contributions.

1. **Two conditions for faithful auditing** (Section 2, Definition 3, Definition 4): set-locality and reseed-identifiability of the policy contrast are two conditions we identify as governing when secret set-dueling can faithfully audit a learned controller—we argue each is necessary and demonstrate the boundary, leaving a full sufficiency theorem to future work (statelessness is one sufficient special case of the second). On a real simulator we probe the boundary one trace at a time: replacement is set-local (and we expose where its stateful reward is not reseed-identifiable), while prefetching is not set-local.
2. **The mechanism** (Section 5): a counterfactual-free competence estimator that repurposes the set-dueling substrate Qureshi et al. (2007); Jaleel et al. (2010) from *choosing* a policy to *trusting* one, gated by a change detector and a four-state trust machine, all off the memory-access critical path (out of the latency-critical access loop).
3. **Two guarantees** (Section 7): an *expected*-regret safety floor (Lemma 1)—bounding the mean regret against the fallback—tied to the run-length theory of the detector, and a mimicry-resistance result (Proposition 1) purchased by the secrecy and per-epoch independence of the sampler—an expectation-level floor plus an exponentially-decaying bound on *sustained* realized evasion; together with an honestly named limitation, a slow-bleed attacker who stays below the audit's resolution and evades it.
4. **A cheap, released design** (Sections 11 and 12): a two-tier microarchitecture whose auditor fits in a few hundred bytes and sits off the cache's critical path, evaluated by a synthetic harness that validates the guarantees and a study on a real cycle-accurate CPU simulator (ChampSim) that both demonstrates the downside cap and exposes the reward-localizability boundary.

We are explicit throughout (Sections 10 and 14) about what is *proven*, what is *measured* in a controlled synthetic harness—whose load-bearing assumptions (bounded reward, a partitionable resource, and a secret uniform sampler, A1–A6; plus, for faithful auditing, set-locality and reseed-identifiability) are exactly those the guarantees rest on—and what is *demonstrated* on a real simulator.

2 Formal setting

A learned controller C is invoked at decision points $t = 1, 2, \ldots$. At each point it observes microarchitectural state $x_t \in \mathbb{R}^d$ (the hardware's internal counters and flags), emits action $a_t \in \mathcal{A}$, and receives a bounded

reward $r_t \in [0, r_{\max}]$ after a short horizon. Examples include prefetch hit×timeliness, cache hit, a normalized latency-saving reward, and branch correctness. The controller was trained or validated on a distribution D_{val} of (x, a, r). A fixed safe fallback π_0 has a known, attack-resistant competence floor.

Definition 1 (Competence advantage)**.** *Index audit windows by w, and let W_w be the decision points in window w. For policy π, define the normalized window reward $\bar{r}_w^{\pi} := |W_w|^{-1} \sum_{t \in W_w} r_t^{\pi}$. The competence advantage is $\Delta_w := \bar{r}_w^{C} - \bar{r}_w^{\pi_0}$.*

$r_t^{\pi_0}$ is counterfactual—one cannot run both policies on the same decision—which is the central estimation challenge Section 5 solves. Unless stated otherwise, the CUSUM, gate, and regret bound below use audit-window units: rewards are the normalized $\bar{r}_w$, and durations are numbers of windows.

Definition 2 (Off-policy condition)**.** *The system is* off-policy *in window w iff $\Delta_w < \tau$ for a trust threshold $\tau \geq 0$. τ=0 means "learned no longer beats the floor"; τ>0 gives margin and hysteresis.*

This one scalar unifies the two threats: an adversarial push toward C's decision boundary and a benign distribution shift both manifest as Δ_w falling. The auditor detects the condition; a lightweight triage (Section 8) guesses the cause to pick the response.

Definition 3 (Set-locality)**.** *Let the resource be partitioned into dueling sets $\{s\}$. A controller is* set-local *if the reward r_t for a decision made on set s is realized on set s (so it can be attributed to s's pool). This is the* spatial attribution *property: each pool mean aggregates only rewards its own policy earned. Set-locality alone does* not *make $\hat{\Delta} = \bar{r}_L - \bar{r}_F$ a faithful competence estimate—faithfulness additionally requires the reseed-identifiability condition of Definition 4 (a set-local, stateful reward can still defeat the reseeded audit, as the real replacement study shows).*

Set-locality prevents $\hat{\Delta}$ (Section 5) from mixing rewards earned by different sets. It is necessary for spatial attribution, but not sufficient for a faithful competence estimate. Cache *replacement* (deciding which cached block to evict) satisfies the spatial-attribution requirement exactly (the eviction on s is rewarded by hits/misses on s); PC-indexed predictors satisfy it too—though, as Definition 4 and Section 11.14 show, replacement's reward is *stateful*, so a secret reseeded audit does not recover a faithful $\hat{\Delta}$ there at per-window grain; *prefetch* violates it (a prefetch on s benefits a different set s'), and we show in Section 11.14 that no per-set reward—not even the controller's own— recovers a faithful $\hat{\Delta}$ for it. We therefore present Bouncer as a mechanism for the *set-local* class, with replacement as the canonical instance and prefetching as the demonstrated boundary.

These are controller-side attribution conditions, not a substitute for statistical representativeness. The pool estimator targets the decision-weighted Δ_w only when the sampled sets represent the domain's traffic, or when the estimator is weighted accordingly. Proposition 1 states this requirement as within-domain exchangeability up to bias β; the synthetic harness enforces it by drawing the same number m of decisions per set. Without such a premise, even a stateless, set-local reward can yield a biased $\hat{\Delta}$ under skewed traffic.

Definition 4 (Reseed-identifiability)**.** *Let $\mathcal{D}_{rs}$ denote the short-run state distribution induced by reseeding leaders. Given representative sampling, define the signed reseed contrast $\hat{g} := \mathbb{E}_{s \sim \mathcal{D}_{rs}}[r_C(s) - r_F(s)]$. A set-local audit is* reseed-identifiable *at threshold τ when reseeding preserves the trust decision,*

$$\operatorname{sign}(\hat{g} - \tau) = \operatorname{sign}(\Delta - \tau),$$

with enough separation from τ for the detector to resolve the decision.

Stateful reward can violate this condition: a cache hit on set s depends on the blocks retained by earlier evictions, while reseeding repeatedly returns sampled sets to short-run state. The signed definition matters because a contrast moved to the wrong side of τ would reverse the gate.

Statelessness removes this state-history failure mode, but it does not remove the representativeness requirement above. Nor is statelessness necessary. An additive warmup $r_C(k) = \mu_C - be^{-ck}$, $r_F(k) = \mu_F - be^{-ck}$ has the state-invariant contrast $\mu_C - \mu_F$ and remains identifiable. A multiplicative warmup $r_C(k) = \mu_C(1 - e^{-ck})$ has contrast $(\mu_C - \mu_F)(1 - e^{-ck})$, which reseeding attenuates (Section 11.7). The balanced, independent

synthetic harness satisfies both representativeness and reseed-identifiability. Real cache replacement is set-local but exhibits the multiplicative failure: reseeded estimates collapse while fixed leaders recover the gap (Section 11.14).

3 Threat model

In scope. A co-located tenant/process sharing C that issues arbitrary memory accesses and knows the Bouncer design (Kerckhoffs). Its goal is denial of service (DoS) or steering C into a covert or side channel.

Out of scope. Physical and fault attacks, direct weight tampering, and online-learning poisoning are outside the base model, which fixes weights at deployment. GATED freezes learning and stops further updates, but it neither repairs a policy poisoned during TRUSTED operation nor prevents PROBING from restoring it. Defending learned *state*, rather than the steered-input threat studied here, remains future work.

The secret. A hardware random-number generator (RNG) supplies a per-epoch seed that assigns the dueling sets. The assignment is not exposed through an architectural channel.

The adaptive adversary (headline). The headline *mimicry* attack keeps Tier-A's input statistics in-distribution while degrading C. Measuring realized competence, rather than inputs, is what exposes this attack; the 50/50-versus-0/50 experiment isolates that distinction (Section 11.5). Secrecy addresses a separate move: an attacker who identifies the dueling sets can spare them and pass the audit. The leak ablation measures this secrecy dependence.

Realizability—what we do and do not demonstrate. Bouncer's guarantees bound the cost of *any* in-domain competence degradation *however it arises*: benign distribution shift or adversarial steering are the same scalar event ($\Delta_w < \tau$) to the auditor. We realize the mimicry/steering adversary in the synthetic competence harness, where the latent stress u is the controllable knob; the real-ChampSim study (Section 11.14) drives the auditor with an *injected, controlled, labelled* competence degradation—a deliberately-bad eviction policy active on a known phase—*not* a demonstrated end-to-end real-silicon steering attack. Whether a co-located tenant can realizably steer a *specific deployed* controller to sub-resolution harm on real silicon, rather than in our harness, is *not established here and is future work*; the defense's guarantee is degradation-source-agnostic, so the harness's injected degradation is a faithful stand-in regardless of how a real degradation would be induced.

4 Architecture

Bouncer is two-tier because cheap input signals are spoofable and direct competence confirmation costs more. Cheap, always-on tripwires (Tier-A) raise the duty cycle of a competence confirmer (Tier-B; Figure 2). The estimator and gate logic are epoch-grained and off the cache-access critical path; the proposed per-access pool-tag lookup and output multiplexer are accounted for separately in Section 12.

Gate state machine. A four-state finite-state machine (FSM) determines where C runs. *TRUSTED* runs C everywhere and Tier-B at a low background duty cycle; a Tier-A cumulative-sum (CUSUM) detector can move it to *SUSPECT*. *SUSPECT* still runs C everywhere but raises Tier-B's duty cycle; a Tier-B alarm moves it to *GATED*, while T_{clear} clear windows restore TRUSTED. *GATED* runs π_0 except on Leader-L, preserving a measurable $\hat{\Delta}$, and freezes online learning. After T_{dwell} windows it enters *PROBING*, which re-enables C only on a small secret audited region. PROBING restores TRUSTED after $T_{reprobe}$ estimates satisfy $\hat{\Delta} \geq \tau + \Delta_{hys}$; otherwise it returns to GATED with exponential dwell backoff. The entry threshold τ, exit threshold $\tau + \Delta_{hys}$, and minimum dwell prevent thrashing and make re-trust measured rather than timer-based.

A design subtlety the build surfaced (Figure 3): a *pure* mimicry attack keeps Tier-A silent, so if SUSPECT were the only route to GATED the attack would evade the gate. We therefore allow the background Tier-B CUSUM to drive TRUSTED $\rightarrow$ GATED directly; Tier-A escalation only *accelerates* detection by raising Tier-B's duty cycle. The two tiers trade latency/energy, not correctness.

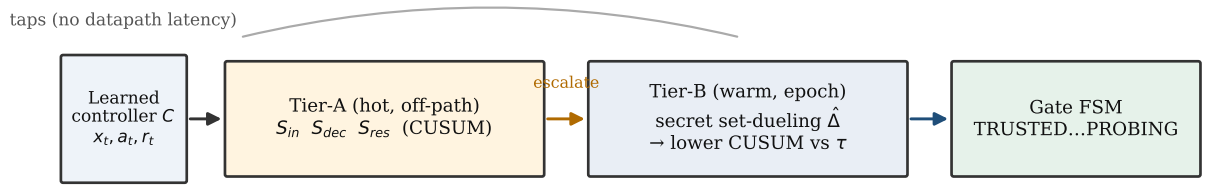


Figure 2: Two-tier auditor. Always-on Tier-A tripwires raise the duty cycle of the Tier-B competence confirmer. The estimator and gate logic are epoch-grained; the per-access pool-tag lookup and output multiplexer remain (Section 12).

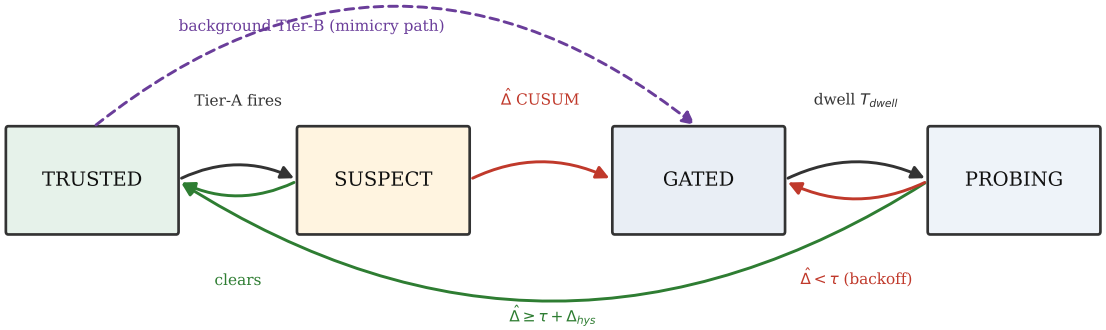


Figure 3: The trust-gate state machine. Hysteresis and a minimum dwell prevent thrash; PROBING makes re-trust measured, not timed. The dashed path lets background Tier-B gate a pure-mimicry attack that keeps Tier-A silent.

5 Tier-B: competence via randomized-secret set-dueling

Mechanism. Partition the controller's resource (cache sets, memory banks, PC-hash buckets) into three pools, reseeded each epoch (a fixed-length run of decisions) from a hardware RNG (random-number generator) whose assignment is unobservable to software: *Leader-L* (n_L sets, always run by C), *Leader-F* (n_F sets, always run by π_0), and the *Follower* rest (run whichever leader currently wins—what the victim mostly experiences). Set-dueling—racing two policies on small dedicated sample pools—is already in silicon Qureshi et al. (2007); Jaleel et al. (2010); Bouncer repurposes it from policy selection to policy trust. The estimator is

$$\hat{\Delta}_w = \bar{r}_{L,w} - \bar{r}_{F,w}, \tag{1}$$

the mean reward rate on the sampled L and F sets in audit window w. Because Leader-L sets *ran* C and Leader-F sets *ran* π_0, $\hat{\Delta}$ is a *measured*, non-counterfactual quantity.

Concentration → detection latency (the key tradeoff). Suppose each set contributes m_{eff} effectively independent bounded reward samples per window and sampled-set means are independent across the two

pools. Then $\mathrm{Var}(\bar{r}_{\mathrm{pool}}) \le r_{\max}^2/(4m_{\mathrm{eff}}n)$, so

$$\mathrm{Var}(\hat{\Delta}_w) \;\le\; \frac{r_{\max}^2}{4m_{\mathrm{eff}}}\Big(\tfrac{1}{n_L}+\tfrac{1}{n_F}\Big) \;=:\; \sigma_\Delta^2. \tag{2}$$

The i.i.d. synthetic harness has $m_{\mathrm{eff}} = m = 64$; correlation reduces m_{eff}, and boundedness alone does not provide the $1/(mn)$ reduction. To resolve a margin around τ one needs σ_Δ smaller than that margin. Smaller pools or shorter windows give a noisier $\hat{\Delta}$, a higher CUSUM threshold H to hold the false-alarm rate, a longer detection delay D, and a larger safety- floor slack (Lemma 1). This chain—*pool size → estimator variance → CUSUM threshold → detection delay → regret floor*—is the central design dependency; every hyperparameter lives on it.

Detector. Let $\gamma > 0$ be a design margin and set the reference value $K = \tau + \gamma/2$. A one-sided lower cumulative-sum (CUSUM) detector on $\hat{\Delta}_w$ catches a downward shift below τ:

$$C_w^- = \max\big(0,\, C_{w-1}^- + (K - \hat{\Delta}_w)\big), \tag{3}$$

alarming when $C_w^- > H$, and resetting on alarm (repeated CUSUM, so it re-arms for the next change rather than latching after a long excursion). Standard quickest-detection theory Lorden (1971); Moustakides (1986) motivates the CUSUM as the near-optimal detector for a downward shift under a false-alarm constraint (optimality is exact only for a known post-change law; here $\hat{\Delta}_w$ is an estimated, variance-inflated statistic, so we treat it as a well-motivated detector). Section 7 gives the average-run-length (ARL) and delay constants.

6 Tier-A: cheap tripwires

Tier-A signals update on a sampled $1/k$ of decisions and *never gate on their own*: they only raise Tier-B's duty cycle. Their false positives therefore cost confirmation work, not correctness. The released prototype uses the dense statistics described below; Section 12 reports an illustrative quantized hardware budget rather than a synthesized implementation.

S_{in} **(input covariate shift).** A frozen reference sketch of D_{val} features: a random projection $P \in \mathbb{R}^{d\times p}$ (p=8) with the projected validation mean and standard deviation. Online, it measures the squared standardized displacement of the projected window mean. This statistic is *spoofable*—the adversary can hold its monitored marginals fixed—which is why it cannot be the trust signal.

S_{dec} **(decision-confidence collapse).** If C exposes value/Q/logits, monitor action-entropy collapse, max-Q margin, and (for online learners) temporal-difference-error magnitude (TD-error, the reinforcement-learning surprise signal). Pythia's reward and the confidence signals of SHiP and perceptron (two learned cache/branch controllers) are already computed in silicon Wu et al. (2011); Jiménez & Lin (2001), so S_{dec} is nearly free there.

S_{res} **(innovation residual).** A linear forward model $g : (x_t, a_t) \to \hat{r}_t$ fit on D_{val} ($d+2$ coefficients for the features, action, and bias; 18 at d=16); online residual $e_t = r_t - \hat{r}_t$; two-sided CUSUM (a cumulative-sum drift detector) on e_t. This catches "the world responds differently than at validation" even when input marginals look clean—the architectural analogue of a Kalman innovation (the gap between a predicted and an observed signal). It is the signal that separates a competence-aware monitor from an input-only one.

7 Guarantees

7.1 Safety floor / bounded regret

Assumption 1. (A1) *Each normalized window reward lies in* $[0, r_{\max}]$.

(A2) *A* drop episode *is a maximal run of windows with* $\Delta_w < \tau$. *During each episode, the expected number of* fully-open *windows—C running everywhere in TRUSTED or SUSPECT, including the initial detection delay and any false re-trust excursions—is at most D.*

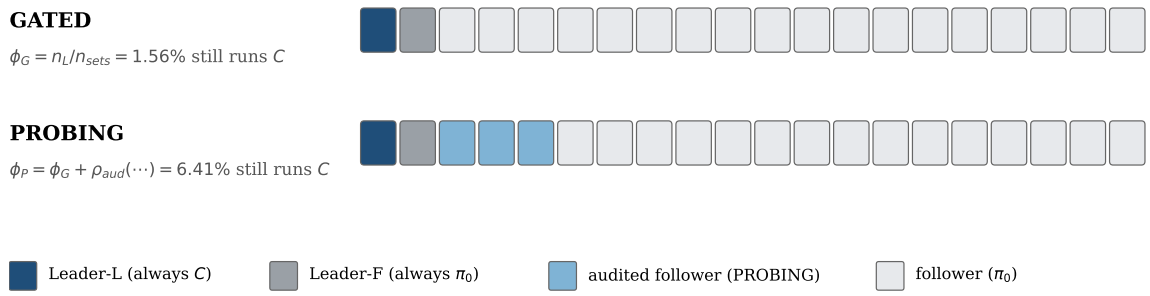


Figure 4: Which sets keep running C during a drop (schematic, not to scale). Leader assignments remain fixed within an epoch and are redrawn at the next. GATED runs C only on Leader-L (ϕ_G); PROBING also runs C on a uniform secret follower subset (ϕ_P total exposure).

(A3) *The marginal false-alarm probability in each window is at most* $\alpha = 1/\mathrm{ARL}_0$*; no independence across windows is assumed.*

(A4) *The total transient regret attributable to one false-alarm episode, including all gate-state transitions it induces, is at most* c_{sw}*.*

(A5) *A horizon of* T *windows contains* N_{ep} *drop episodes spanning* $T_{att} \le T$ *windows.*

(A6) *During a drop, the sets that keep running* C *are chosen by the secret sampler: the* n_L *Leader-L sets always, plus in PROBING a uniform secret follower subset of fraction* ρ_{aud}*, redrawn each epoch. Thus each set carries* C *with probability at most* ϕ_P*, independent of its traffic (*ϕ_G *while GATED and* ϕ_P *while PROBING).*

A2 is an empirical assumption about the finite-state machine (FSM), not a closed-form delay bound. Section 11.12 measures it through the implemented controller. In the reseeded, near-i.i.d. harness, false re-trust is approximately zero and D is close to the initial-delay estimate $H/(K - \Delta_w)$. Temporal correlation can inflate D; this temporal premise is distinct from the signed-mean reseed-identifiability condition of Definition 4.

Audit exposure (from the construction). A fixed slice of the resource keeps running the controller C even during a drop, and this slice sets the audit-exposure cost. The n_L Leader-L sets run C in *every* gate state (they must, so $\hat{\Delta}$ stays measurable and PROBING re-trust is *measured* rather than timed; Section 5), and PROBING additionally re-enables C on a *uniform secret* fraction ρ_{aud} of the followers—drawn by the same secret sampler and redrawn each epoch, so a set's exposure does not depend on its index or traffic (a fixed, e.g. sorted, audit subset would violate this; Section 11.11). So the share of the resource still running C during a drop is $\phi_G = n_L/n_{sets}$ while GATED and $\phi_P = \phi_G + \rho_{aud}\left(1 - \frac{n_L+n_F}{n_{sets}}\right)$ while PROBING; at the pinned operating point these shares are small (ϕ_G=1.56%, ϕ_P=6.4%). Let T_{att} be the number of windows inside drop episodes.

Lemma 1 (Safety floor, in expectation)**.** *Under Assumption 1, for an* arbitrary *per-set traffic distribution, the expected regret against the fallback—over the secret sampler, the detection delay, and the false-alarm events—obeys*

$$\mathbb{E}\Big[\sum_{w=1}^{T}\big(\bar{r}_w^{\pi_0} - \bar{r}_w^{\mathrm{Bouncer}}\big)\Big] \;\le\; \underbrace{N_{ep}\,D\,r_{\max}}_{\text{detection + re-trust}} + \underbrace{\phi_P\,T_{att}\,r_{\max}}_{\text{audit exposure}} + \underbrace{\alpha\,T\,c_{sw}}_{\text{false alarm}}\,. \tag{4}$$

The bound uses A1–A6 and linearity of expectation; its counting arguments require only the marginal conditions in A2, A3, and A6, not independence across windows. The middle term is the price of continuous, measured *auditability; it grows only at the dueling fraction ϕ_P of the attack duration, not at full resource.*

Proof sketch. Partition windows and take expectations termwise.

(i) Audit exposure. During a drop, Leader-L and the uniform secret PROBING subset still run C. Their regret depends on the *traffic* on those sets, not their count. A hot set tagged for C can therefore cost a full $r_{\max}$ in one window (Section 11.11). Under A6, however, each set carries C with probability at most ϕ_P, independent of its traffic. For any fixed traffic vector q with $\sum_s q_s = 1$, $\mathbb{E}[\text{exposure}] = \sum_s q_s \Pr[s \text{ runs } C] r_{\max} \leq \phi_P r_{\max}$. Summing over drop windows gives $\phi_P T_{att} r_{\max}$.

(ii) Detection and re-trust. A2 counts every fully-open window in an episode, including false re-trust excursions. Their expected regret is therefore at most $N_{ep} D r_{\max}$.

(iii) False alarms. By A3, the expected number of false-alarm episodes is $\sum_{w=1}^{T} \Pr[\text{alarm}_w] \leq \alpha T$; linearity suffices. A4 bounds the total transient regret of each episode by c_{sw}.

(iv) Windows in which the open controller outperforms the fallback contribute non-positive regret. Summing (i)–(iii) proves eq. (4). □

A high-probability refinement, for exposure only. For a fixed sequence of T_{att} drop windows, let $X_w \in [0, r_{\max}]$ be exposure regret. Traffic may adapt to gate history, so the X_w need not be independent. Under A6, however, $\mathbb{E}[X_w \mid \mathcal{F}_{w-1}] \leq \phi_P r_{\max}$: the current secret draw is uniform and independent of the past, and the input-only adversary is blind to it. Thus $X_w - \mathbb{E}[X_w \mid \mathcal{F}_{w-1}]$ is a martingale difference whose conditional range has width $r_{\max}$. Hoeffding–Azuma gives, with probability at least $1 - \delta_s$,

$$\sum_w (\text{exposure regret})_w \;\leq\; \phi_P T_{att} r_{\max} \;+\; r_{\max} \sqrt{\tfrac{T_{att}}{2} \ln \tfrac{1}{\delta_s}}, \tag{5}$$

an $o(T_{att})$ slack that vanishes per-window over a sustained attack (empirically the envelope covers the worst-case concentrated-traffic total with coverage 1.0; Section 11.11). We do *not* claim analogous high-probability forms for the detection and false-alarm terms: the former would need a per-episode delay-*tail* bound (A2 supplies only the mean) and the latter a martingale/renewal concentration for the *stateful* CUSUM (A3 supplies only the marginal), neither of which we assume. The guarantee we assert is the expected-regret bound eq. (4).

Two of the three regret terms are tunable, and one is fixed by design. The detection and false-alarm slack are the Section 5 knobs, while the audit-exposure term is instead fixed by the design constant ϕ_P. In the reseeded regime where false re-trust ≈ 0 (Section 11.12), A2's D reduces to the initial delay $D \approx H/(K - \Delta_{true})$, and the false-alarm rate is $\alpha = 1/\text{ARL}_0$, where ARL_0 (average run length to a false alarm) comes from Siegmund's corrected- boundary approximation Siegmund (1985), a standard sequential-analysis result. For a one-sided CUSUM (a running change detector) accumulating $(K - \hat{\Delta})$ with standardized drift $\delta = (K - \mu)/\sigma_\Delta$ and boundary $b = H/\sigma_\Delta + 1.166$,

$$\text{ARL}(\delta) \;\approx\; \frac{e^{-2\delta b} + 2\delta b - 1}{2\delta^2}. \tag{6}$$

In-control ($\mu \gg K$, $\delta < 0$), ARL_0 grows *exponentially* in H/σ_Δ, so false alarms are rare; out-of-control ($\delta > 0$), the delay is short. The guarantee thus degrades gracefully and *quantifiably* with detection delay and false-alarm rate.

7.2 Mimicry resistance

We state the property *conditionally*—it is exactly as strong as the secrecy and the within-domain exchangeability it assumes, and we are explicit that a *global* $\hat{\Delta}$ carries *no* guarantee against a redistributive attack.

Proposition 1 (Mimicry resistance, per-domain)**.** *Fix a contested domain R, such as a victim's cache footprint. In every epoch, draw Leader-L and Leader-F uniformly without replacement within R; the remaining*

sets are followers, and the assignment is hidden from the adversary. Let $\mu_{0,R} := \mathbb{E}[\bar{r}_{F,R}]$ *be the fallback pool's expected reward. Assume that controller rewards on Leader-L and follower sets are exchangeable up to a bias* $|b_R| \le \beta$*, and that the adversary holds aggregate Tier-A statistics at their validation values.*

(i) Expectation. *Any input-only strategy with* $\mathbb{E}[\hat{\Delta}_R] \ge \tau$ *satisfies*

$$\mathbb{E}[\bar{r}^C_{\text{foll},R}] \ge \mu_{0,R} + \tau - \beta.$$

This is an unconditional expectation statement, not a guarantee conditional on a single realized audit.

(ii) Sustained realized evasion. *Consider a block of* W *windows. Assume the* $\hat{\Delta}_{R,w}$ *are independent conditional on the strategy and that* $W^{-1}\sum_{w=1}^{W}\mathbb{E}[\hat{\Delta}_{R,w} \mid \text{strategy}] \le \tau - \epsilon$*. Fresh secret assignments alone do not imply this temporal premise; gate-history-dependent reward processes can violate it. Because* $\hat{\Delta}_{R,w} \in [-r_{\max}, r_{\max}]$*, for any* $W \ge 2H/\gamma$*,*

$$\Pr[\textit{no alarm in the block}] \le \Pr[W^{-1}\textstyle\sum_w \hat{\Delta}_{R,w} \ge \tau] \le e^{-W\epsilon^2/(2r^2_{\max})}.$$

Proof sketch. (i) Exchangeability gives $\mathbb{E}[\bar{r}^C_L] = \mathbb{E}[\bar{r}^C_{\text{foll},R}] + b_R$. Since $\mathbb{E}[\hat{\Delta}_R] = \mathbb{E}[\bar{r}^C_L] - \mu_{0,R} \ge \tau$ and $|b_R| \le \beta$, rearrangement proves the stated floor. Conditioning instead on $\{\hat{\Delta}_R \ge \tau\}$ would select favorable audit noise (Section 11.13).

(ii) Hoeffding's inequality for independent, not necessarily identical, variables of range $2r_{\max}$ gives the high-average bound $e^{-W\epsilon^2/(2r^2_{\max})}$. On a no-alarm block, no reset occurs and

$$C_w = \max(0, C_{w-1} + K - \hat{\Delta}_{R,w}) \ge \sum_{j=1}^{w}(K - \hat{\Delta}_{R,j}).$$

The strict alarm rule is $C_w > H$, so no alarm gives $C_w \le H$. At the end of the block this implies $W^{-1}\sum_w \hat{\Delta}_{R,w} \ge K - H/W \ge \tau$ when $W \ge 2H/\gamma$. Thus the no-alarm event is contained in the high-average event. □

Partial leakage (empirical scope). The leak ablation uses a homogeneous regional degradation model in which sparing a fraction f of Leader-L sets approximately removes the same fraction of degradation. At the operating point, $\hat{\Delta}_R \approx 0.084 < K$ at f=0.6 and $0.227 > K$ at f=0.8, matching the observed collapse in that band (Section 11.5). This is not a general f-linear theorem: leaked sets could be a nonrepresentative subset of traffic or harm.

What this does and does not cover. (i) *Global* $\hat{\Delta}$ *is not enough.* A purely redistributive adversary that degrades the victim region while holding the *global* mean fixed evades a global $\hat{\Delta}$ outright; only per-domain dueling (the construction above, used in Section 11.5) detects it. (ii) *Heterogeneity bias.* Real cache sets / PC-buckets are not i.i.d. (hot/cold sets, conflict skew), so $\beta > 0$; the guarantee degrades by β. Secret *reseeding* each epoch shrinks this bias toward the i.i.d. value by re-randomizing which physical cache sets act as leaders. (iii) *Secrecy is load-bearing.* The whole argument rests on the adversary being blind to the within-R assignment—which is why the secrecy ablation (Section 11.5) is the headline experiment.

The coverage hole (an explicit slow-bleed attack on the budget). A budgeted auditor has a finest resolvable competence drop. Let $\rho := n_{L,R}/|R|$ be the per-pool leader density, k the chosen standard-error multiplier, and B the available per-pool leader budget. Under the effective- independence premise of eq. (2), the within-R resolvable gap is $\delta_R = k\,\sigma_{\Delta,R} = kr_{\max}/\sqrt{2m_{\text{eff}}\rho|R|}$, requiring $n_{L,R} \ge k^2 r^2_{\max}/(2m_{\text{eff}}\delta^2_R)$ leaders, independent of $|R|$. A fixed budget B therefore induces a finest resolution $\delta^{\min}_R(B)$, and a redistributive adversary can pick a region small enough (or trickle harm slowly enough) that its per-window competence drop stays *below* $\delta^{\min}_R(B)$ forever—an undeclared, sub-resolution "coverage hole" that the safety-floor lemma, proven only over the *union of audited domains*, does not bound. Finite resolution and assignment leakage are distinct failure axes: the former hides small harm even with an intact secret, whereas the latter lets an attacker spare audited sets. *We therefore claim mimicry resistance only for declared domains audited at resolution* $\delta^{\min}_R(B)$ *with an intact secret—not general, any-region security*; the slow-bleed attack is an open

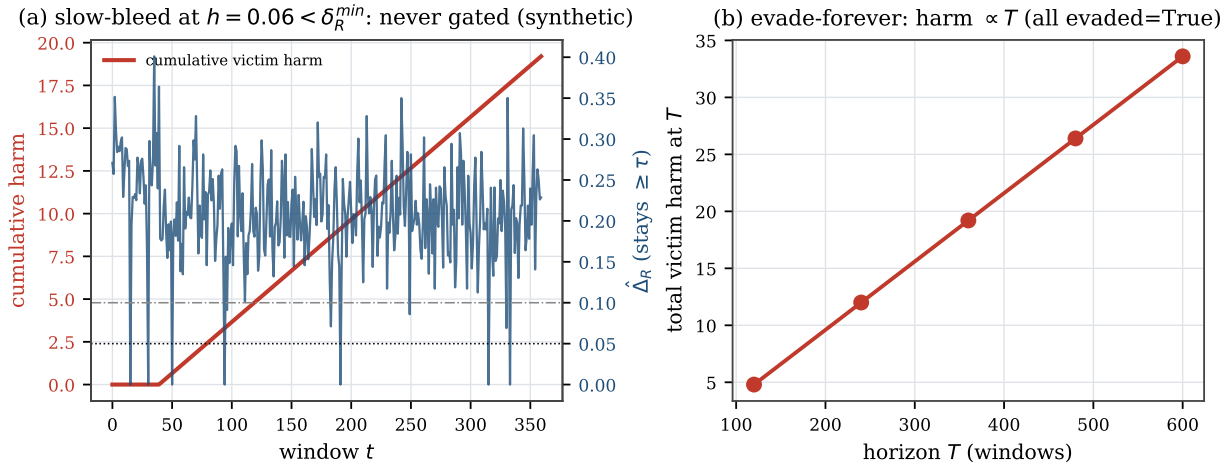


Figure 5: The coverage hole, demonstrated (controlled harness, real Bouncer region dueling). (a) a slow-bleed at $h{<}\delta_R^{\min}$ accumulates unbounded victim harm while $\hat{\Delta}_R \geq \tau$ and no alarm fires; (b) it evades at every horizon, so total harm $\propto T$. The closed-form floor is $\delta_R^{\min}(B){=}k\sigma_{\Delta,R} \propto 1/\sqrt{B}$ $(0.19, 0.13, 0.094, 0.066$ for per-pool leader counts $n_{L,R}{=}2, 4, 8, 16)$: more leader budget shrinks the hole but never closes it.

vulnerability. Figure 5 demonstrates it against the deployed auditor: constant per-window harm $h{=}0.06$ kept below $\delta_R^{\min}$ bleeds a victim region for the entire horizon; cumulative harm grows linearly as the horizon T increases, while the measured gap $\hat{\Delta}_R$ holds above the trust threshold $(0.205 \geq \tau)$ and the CUSUM (the cumulative-sum drift detector) never fires.

8 Attack-vs-drift triage

Attack and drift both trip the same failure test ($\Delta_W < \tau$, the audited competence margin falling below threshold), yet leave distinguishable signatures. Those differing signatures let us set the response prior (a secondary contribution—detection does not depend on it). Drift is slower and phase-marker-correlated (tracking program-phase transitions) and is *self-consistent* with a new-but- valid regime (the forward model S_{res} stays calibrated). An attack is abrupt, can pulse, is *targeted* at the decision boundary (high S_{res} with small S_{in}), and correlates with a specific co-tenant's schedule whose work yields it no benefit except moving the shared C (the contended shared resource). A small logistic model over the auditor-observable $(\Delta S_{in}, \Delta S_{res}, |\nabla\hat{\Delta}|,$ co-tenant- corr) suffices to separate the two. In the evaluation, probe P3 validates this triage model under cross-validation (holding out data to check it generalizes; Section 11); misclassification only mis-selects between equally-safe responses.

9 Per-controller instantiation

Table 1 maps Bouncer onto four controller classes, ordered by *set-locality*. **Cache replacement** leads: its hit/miss reward is attributed to the set whose eviction it decided, giving the required spatial attribution; representative sampling and reseed-identifiability are still needed. **Branch/perceptron** predictors are indexed by program counter (PC), so the decision and correctness reward are structurally local to a PC bucket, but reseed-identifiability on real predictors is untested. **Prefetching** is *not* set-local—a prefetch on set s rewards a different set s', so the faithful reward cannot be dualized (Section 11.14); it is the cautionary boundary. The **memory scheduler** is the hardest: no clean set-sampling on a shared command bus to dynamic random-access memory (DRAM) forces a *model-based* $\hat{\Delta}$, which weakens Proposition 1.

Table 1: Candidate controller instantiations. Set-locality asks whether a decision and its reward share a dueling bucket; real-predictor reseed-identifiability remains untested.

Controller	set-local?	r_t proxy	fallback π_0	dueling resource
Hawkeye/Glider (repl.)	**yes**	hit/miss	RRIP/LRU	cache sets
Perceptron branch	yes (PC)	correct	bimodal/gshare	PC slices
Pythia (RL prefetch)	**no**	acc.×timel.	next-line/off	PC buckets
RL mem. scheduler	partial	−lat./+BW	FR-FCFS	banks/chan.

10 Methodology

The formal results are conditional. Lemma 1 uses A1–A6; Proposition 1 additionally uses within-domain representativeness and a temporal premise; estimator sizing in eq. (2) uses effective independence. Faithful attribution also requires set-locality and reseed-identifiability (Definitions 3 and 4). We map each premise to evidence or assumption below rather than treating the environment as irrelevant.

Controlled quantitative claims—latency, floor, mimicry survival, and secrecy— come from one synthetic harness that satisfies these premises and is instantiated three ways in P3. A separate ChampSim integration runs the auditor as a level-1 data-cache prefetcher and last-level-cache replacement module on SPEC CPU2017 traces. It is a real-simulator boundary study, not a full benchmark sweep (Section 11.14).

The harness uses n_{sets}=2048 sets and m=64 independent decisions per set per window. An unobserved stress $u \in [0,1]$ drives drift and steering through $\mu_C(u) = r_{\max}\,\mathrm{clip}(0.8 - 0.7u)$; the fallback is $\mu_0 = 0.5 r_{\max}$, or $q_0 := \mu_0/r_{\max} = 0.5$. The map $\mathrm{IPC}(\bar{r}) = 0.6 + 1.4\bar{r}$ converts normalized reward to instructions per cycle (IPC). Section 11.8 varies both maps and $q_0 \in [0.4, 0.6]$; only the tested qualitative behavior, not a universal transfer-function claim, is asserted.

The operating point is n_L=n_F=32, m=64 ($|W| \approx 1.3\times10^5$ decisions), τ=0.05, K=0.10, and H=0.8. At the boundary, CUSUM drift is $1.6\,\sigma_\Delta$ on the background pool (n_{bg}=8) and $3.2\,\sigma_\Delta$ at full duty (n_L=32). The R-latency sweep therefore reports detection latency and fixed-deadline true-positive rate rather than claiming deadline-free power (Figure 16). Conditions include clean behavior, benign drift, and broad, mimicry, and regional attacks. Baselines are no auditor, input out-of-distribution (OOD) monitoring through S_{in}, oracle-Δ, and always-fallback.

Hypothesis discharge. Table 2 maps each premise to where it is established, measured, or assumed. It also records which premises the real-simulator studies do not satisfy.

11 Evaluation

The evaluation has four blocks. P0–P2 validate the estimator, causal gate, and cost trade-off at the pinned synthetic operating point. P3–P4 test transfer across synthetic controller abstractions and the constructed mimicry/leak attacks. The following stress tests isolate assumptions and failure boundaries. We finish with the formal regressions and the ChampSim boundary studies.

11.1 P0: the estimator tracks ground truth

The estimator $\hat{\Delta}_w$ tracks the ground-truth competence gap Δ_w closely as a benign drift carries competence from positive through zero to negative (Figure 6(a)). The fit is R^2=0.997, with root-mean-square error $0.0141 \approx \sigma_\Delta$=0.0156. This is a *controlled-harness* fidelity at the synthetic operating point (m=64); on a real trace with far fewer samples per bucket the estimator is correspondingly noisier (Section 11.14). Figure 6(b) checks the concentration bound eq. (2) in the i.i.d. harness: across a pool-size sweep the empirical std of $\hat{\Delta}$ stays just below the bound at 0.85–0.98× it, a tight upper envelope (the bound uses $\frac{1}{4} \geq p(1-p)$).

Table 2: Hypothesis discharge: where each premise is established (E), measured (M), or assumed (A). The harness satisfies the statistical premises by construction; the real-cache study measures a reseed-identifiability failure.

Premise	Discharged
A1 bounded reward $\in [0, r_{\max}]$	E/M: synthetic by construction; real hit/miss $\in \{0, 1\}$.
Representative within-domain sampling	A/E: equal m per set in the harness; required, or replaced by traffic weighting, for $\hat{\Delta}$ to target decision-weighted Δ_w.
Effective independence for eq. (2)	A/E: i.i.d. Bernoulli draws in the harness give $m_{\text{eff}} = m$; correlated streams require a smaller effective sample size.
Reseed-identifiability of the contrast (Definition 4)	A (synthetic: i.i.d. reward, holds by construction); **fails, measured, on a real cache** (Section 11.14: reseeded $\hat{\Delta} \approx 0$ vs fixed-leader 0.23).
A2 *expected* fully-open windows/episode $\leq D$	A (assumed; $H/(K-\Delta)$ is a mean-delay approximation, not an upper bound); M: P1, R-latency, and false re-trust (Section 11.12).
A3 marginal false-alarm prob. $\leq 1/\text{ARL}_0$	A (assumed; *approximated* by Siegmund's ARL, the mean run length between false alarms); M: P2 ROC (true- vs false-positive curve), sensitivity sweep.
A4 false-alarm-episode transient $\leq c_{sw}$	A (analytical accounting assumption).
A5 N_{ep} episodes over T	A (definitional, counts episodes).
A6 secret *uniform* audit subset	E: construction (Section 5); M: PROBING exposure is ϕ_P-uniform, index-independent (Section 11.11: 0.062/0.064 low/high index vs 0.985 for a sorted prefix).
Prop. 1(ii) temporal premise (per-window $\hat{\Delta}_R$ independent given the strategy)	A: holds in the synthetic harness (i.i.d. per-decision reward + fresh reseeds); **violable** by gate-history-dependent adversaries and correlated audits (Section 11.12); *not* inferable from reseeding alone.
Prop. 1 finite resolution $\delta_R^{\min}(B)$	E under eq. (2)'s effective-independence premise; M: sub-resolution slow-bleed (Figure 5).
Prop. 1 intact secret (leak f=0)	A in the proposition; M: homogeneous-degradation leak ablation, with collapse in the f=0.6–0.8 band.

11.2 P1: detection, steady floor, and re-trust

The attack begins at window 90 and ends at 200. The unguarded controller falls well below the fallback, while Bouncer detects after **two** causally routed audit windows and then uses π_0. After the attack ends, PROBING re-trusts C after 15 windows; this recovery is measured rather than timer-only.

During attacked GATED windows, only the Leader-L fraction $\phi_G = 32/2048$ continues to run C. The predicted mean IPC-floor violation is therefore

$$\phi_G \big(q_0 - \mu_C^{att}/r_{\max}\big) \frac{1.4}{\text{IPC}(q_0)} = 0.579\%,$$

where $q_0 = 0.5$, $\mu_C^{att}/r_{\max} = 0.156$, and $\text{IPC}(q_0) = 1.3$. The measured attacked-GATED mean is 0.580% and the worst window is 0.64%. This is a mechanism calculation under the synthetic IPC map, not a universal floor predictor.

The two pre-gate windows produce the larger transient dip (36.5% at its worst). That cost belongs to the fully-open term in Lemma 1. Once gated, performance returns to 0.55: the fallback floor, not clean performance, because the attack removed C's value. On clean input, the performance ratio is 0.9965; the permanent cost is the Leader-F sets that run π_0. The loose three-term expected-regret plug-in is 12.4 IPC·windows. The smaller realized-gap/occupancy expression is reported later only as a descriptive tracker, not as a bound.

11.3 P2: ROC, the §5 chain, and overhead

The competence detector catches every tested attack while almost never firing on benign episodes. Figure 8 shows a true-positive rate (TPR) of 1.0 at false-positive rate (FPR) at most 0.05 on the broad and constructed mimicry attacks, while the input-OOD detector S_{in} works on the broad attack but collapses to TPR=0 under mimicry. Shrinking the dueling pool trades measurement noise for detection latency, tracking the analytic $D \approx H/(K-\Delta)$. Figure 9(a) traces the Section 5 chain: as the pool shrinks from 128 to 2, σ_Δ rises

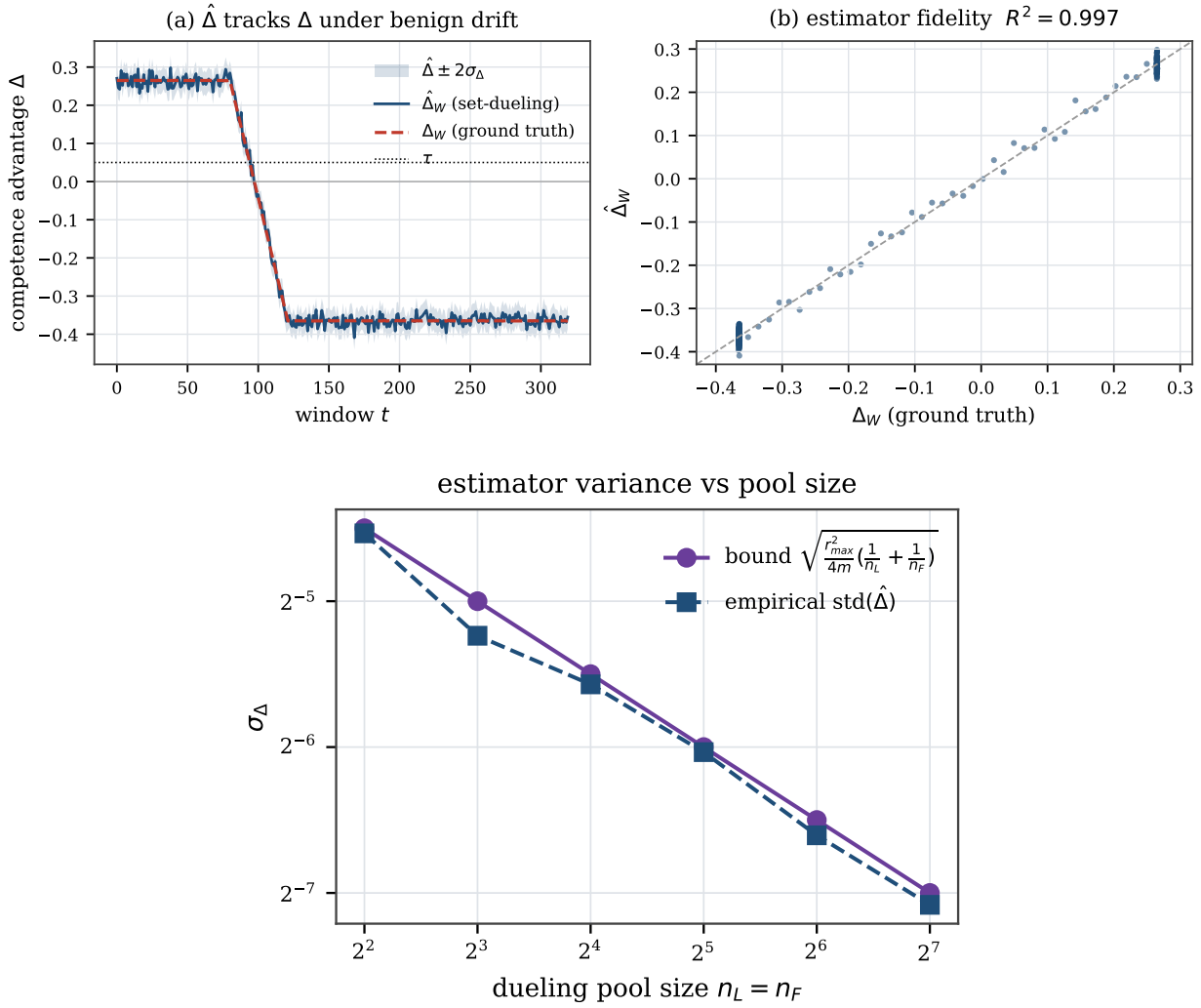


Figure 6: P0. (top) $\hat{\Delta}$ tracks ground-truth Δ (R^2=0.997); (bottom) empirical std($\hat{\Delta}$) remains just below the variance upper bound eq. (2) in the i.i.d. harness.

0.016 → 0.125, the calibrated H rises 0.05 → 0.35, and detection latency rises 0 → 3.4 windows. There is a clear knee near $n \approx 8$: above it detection is gap-limited and pool-insensitive, so larger pools only add overhead; Figure 9(b) is the resulting overhead/latency Pareto frontier. The illustrative 16-bit storage budget in Section 12 is 404 bytes, or 0.154% of a 256 KB SRAM. It is not an RTL area, energy, or timing result.

11.4 P3: synthetic controller abstractions and triage

In the controlled harness, we instantiate the same auditor around abstract prefetcher-, replacement-, and scheduler-like controllers (Figure 10(a)). The first two use secret set-dueling; the scheduler uses a model-based $\hat{\Delta}$. Across these abstractions, the steady-state floor violation remains at most 0.7% under broad attack and drift. The scheduler's weaker estimator increases latency and does not inherit Proposition 1.

The constructed attack and drift episodes are also separable with auditor-observable features (Figure 10(b)). Five-fold cross-validation over 120 episodes reaches 100% accuracy; abruptness is computed from estimated $\hat{\Delta}$, not ground truth. Accuracy remains 100% when any one feature is removed. This establishes separability in the evaluated synthetic episode family, not general attack attribution.

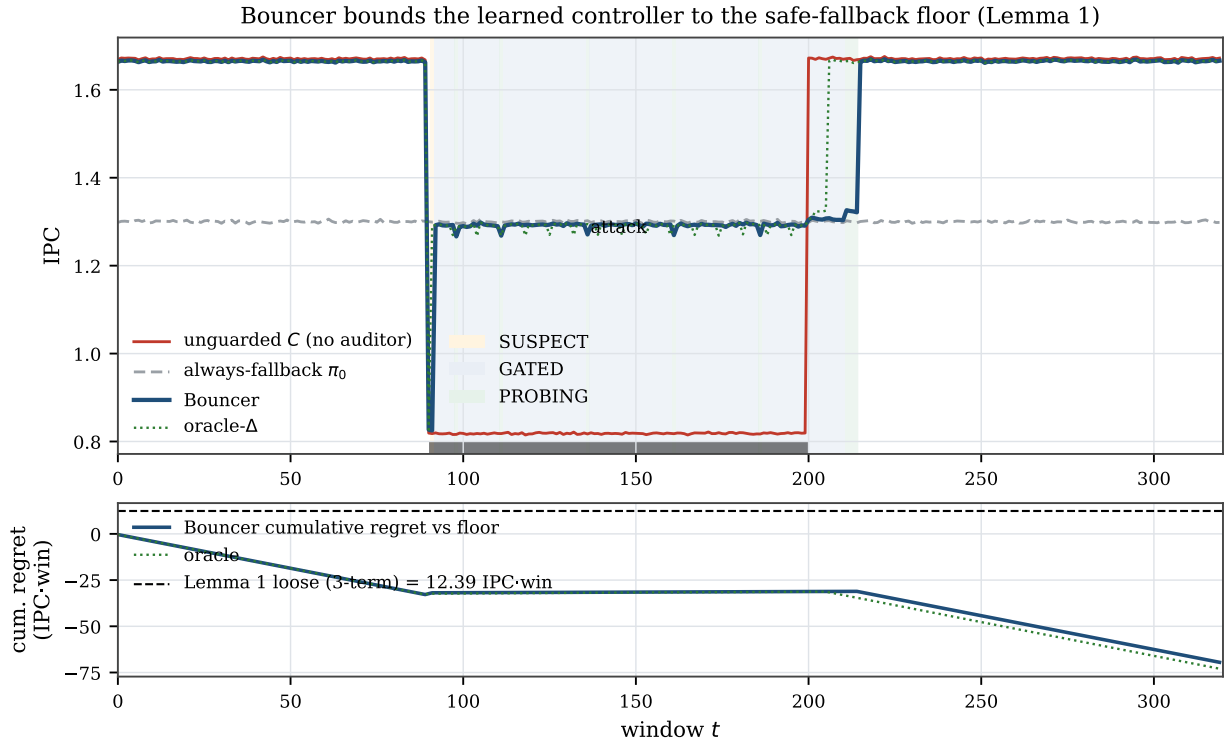


Figure 7: P1. Bouncer bounds the controller to the fallback floor under attack and re-trusts after it ends; the unguarded controller crashes below the floor.

11.5 P4: mimicry survival and the secrecy ablation

Competence versus input monitoring. The mimicry adversary holds S_{in} in-distribution while degrading C. At each tested strength, the input-OOD baseline detects 0/40 episodes and full Bouncer detects 40/40, with 0/40 clean false alarms for both. The input monitor's failure follows from the construction (`suppress_features`); Bouncer's measured 40/40 is the empirical result. Across 50 seeds, Bouncer's true-positive rate (TPR) is 1.0 with Wilson 95% interval $[0.93, 1.0]$, versus input-OOD TPR 0.0 with interval $[0, 0.07]$. Both clean false-positive rates (FPRs) are 0.0 $[0, 0.07]$. Detection latency is 2.0 windows $[2.0, 2.0]$, steady floor violation is 0.641% $[0.638, 0.643]$, and clean performance ratio is 0.99653.

What secrecy adds. The regional experiment approximates Proposition 1's fixed, nonempty within-R pools by intersecting global pools with R. This gives about four in-region Leader-L sets; a resulting pool is empty with probability 0.013 per window (0.027 for either pool), in which case $\hat{\Delta}_R$ is set to zero. Under this approximation and homogeneous regional degradation, TPR is 1.0 for leak fraction $f \leq 0.4$, 0.975 at f=0.6, 0.075 at f=0.8, and 0 at f=1.0; latency rises from 2.2 windows to infinity. Thus panel (a) isolates competence measurement, while panel (b) isolates secrecy in this tested regional model. It is empirical evidence, not a proof of an f-linear leak law.

11.6 Robustness: sensitivity and confidence

The headline numbers are not single-seed artifacts (Figure 12(a), N=50, Wilson 95% intervals above), nor a knife-edge operating point: sweeping the trust threshold τ and CUSUM threshold H over a 6×6 grid (Figure 12(b)), most cells achieve ≤ 3-window detection with clean FPR ≤ 5% and no missed episodes (**22/36**)—a broad plateau around the chosen (τ=0.05, H=0.8). Detection degrades gracefully and predictably toward the plateau edges (small H raises false alarms; large τ raises misses), exactly as the Section 5 chain predicts.

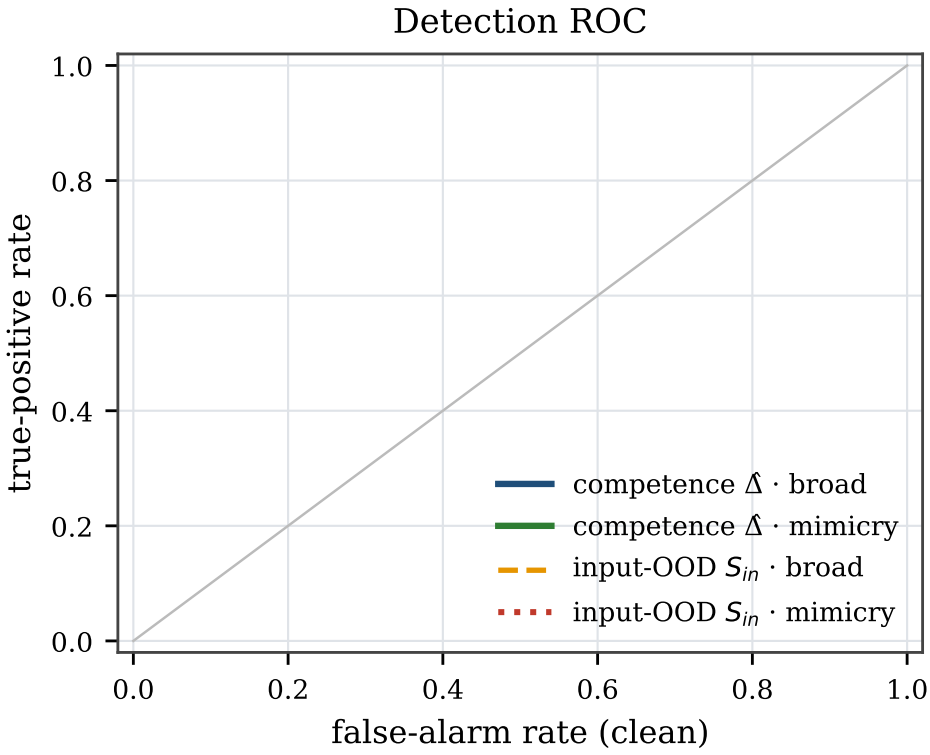
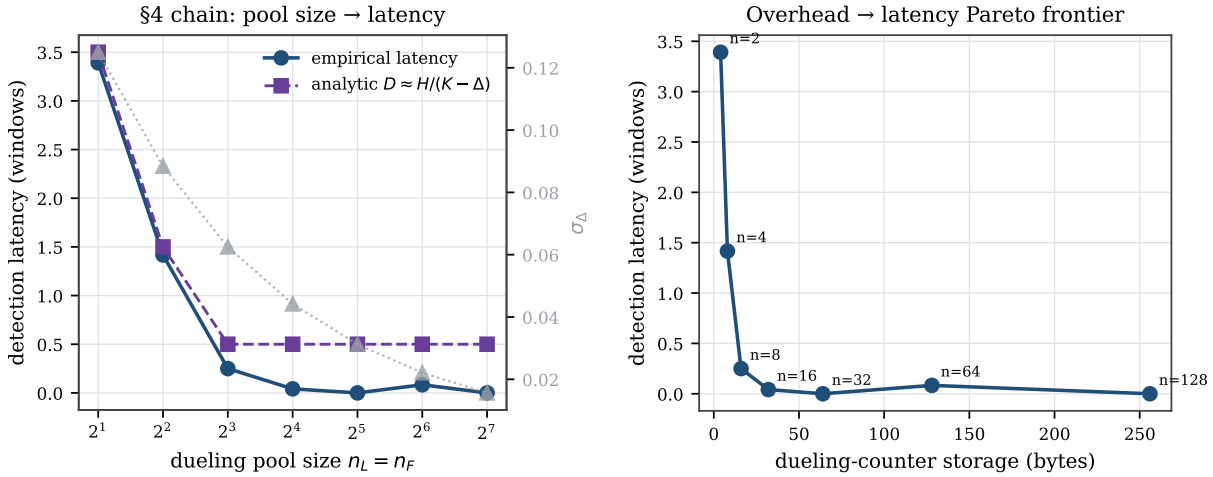


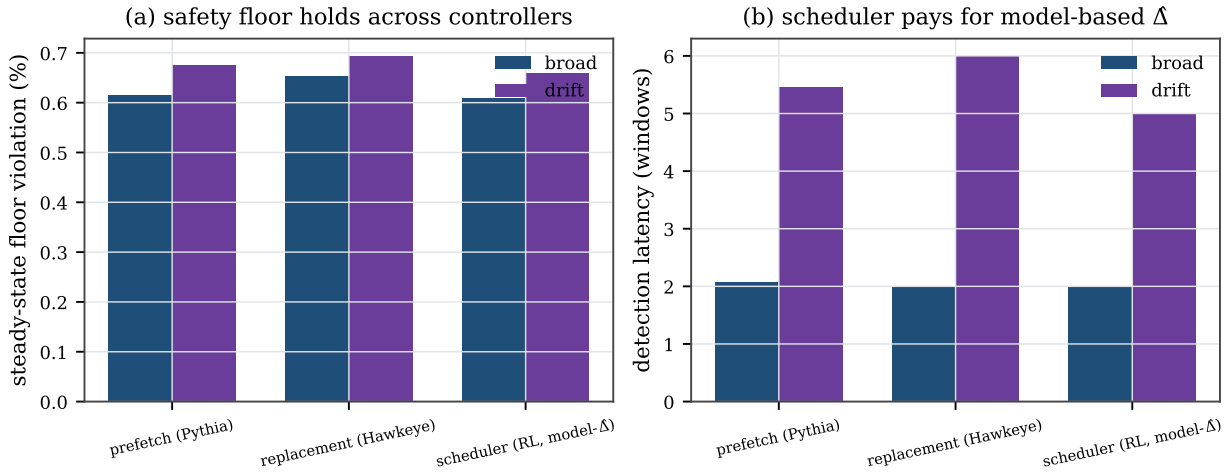
Figure 8: P2 ROC. The competence detector dominates; S_{in} is blind to mimicry.



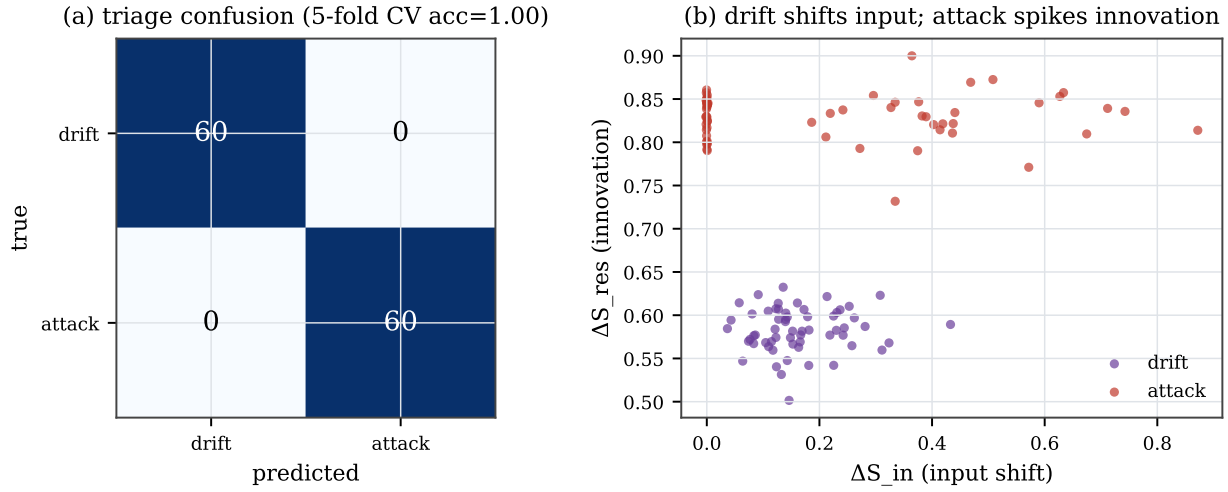
(a) (b)

Figure 9: P2. (a) pool size → σ_Δ → H → latency, with a knee near n=8; (b) overhead/latency Pareto frontier.

Misspecification robustness. Because the harness reward is built from the theorems' assumptions (bounded reward, near-i.i.d. sets)—i.e., a reward specification that may not match reality—we test whether the results survive violating them. We give each set a fixed competence offset $\sim \mathcal{N}(0, \sigma_{het})$ (hot/cold sets, conflict skew), so Leader-C and Leader-F pools are exchangeable only in expectation—the β-bias regime of Proposition 1. Sweeping σ_{het} from 0 to 0.30 (Figure 13), mimicry detection stays at TPR 1.0 and



(a)



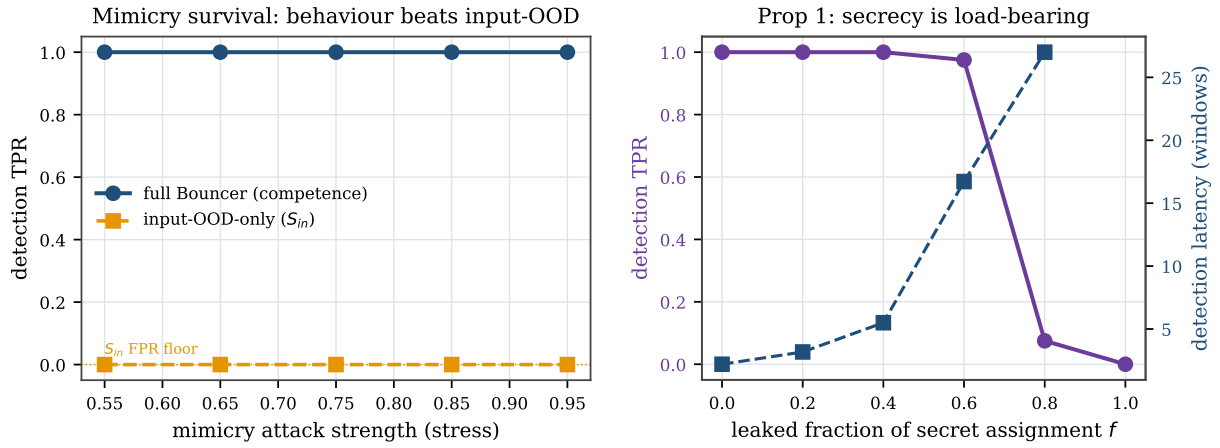
(b)

Figure 10: P3 synthetic abstractions. (a) The floor holds across three harness instantiations; (b) attack and drift are separable within the constructed episode family (five-fold accuracy 1.0).

clean FPR at 0, the steady floor holds (0.60–0.64%), and detection latency degrades only gently ($2.0 \to 2.7$ windows)—because secret *reseeding* re-randomizes which physical sets are leaders each epoch, averaging the heterogeneity bias down. This supports robustness to the tested fixed cross-set heterogeneity. It does not test temporal dependence or remove the effective-independence premise of eq. (2).

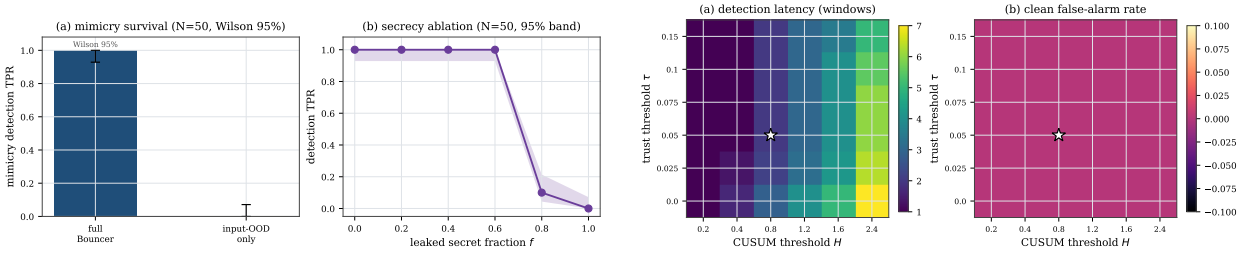
11.7 Reseed-identifiability: a stateful reward that stays auditable

Definition 4 predicts that a secret reseeded audit fails not on *every* stateful reward, but only when the policy *contrast* depends on the state the reseed erases. We isolate the two regimes on a predictor-like controller whose accuracy ramps with consecutive windows on a policy (Figure 14). With *multiplicative* warmup ($\text{acc} = \mu(1 - e^{-ck})$, so the contrast scales with the warmup state), the secret per-epoch reseed makes almost every sampled slice cold and $\hat{\Delta}$ attenuates from 95% to 5% of the fixed-leader estimate ($0.285 \to 0.015$ over $1/c{=}0.3 \to 20$ windows), while *fixed* leaders recover the full gap (0.30). With *additive* warmup ($\text{acc} = \mu - be^{-ck}$, contrast state-invariant at 0.30), the reward is equally stateful yet the reseeded $\hat{\Delta}$ *survives* at 0.30 (± 0.001) across the same warmup range—a stateful reward that stays auditable, the



(a) (b)

Figure 11: P4. (a) In the constructed mimicry attack, Bouncer detects every episode while the input-OOD baseline is blind by construction. (b) Detection in the homogeneous regional model collapses as the secret assignment leaks.



(a) (b)

Figure 12: Robustness. (a) headline results over 50 seeds with Wilson 95% intervals; (b) sensitivity of detection latency and clean false-alarm rate to (τ, H)—the operating point ($\star$) sits on a broad plateau.

direct counterexample to a statelessness *requirement*. The multiplicative case is the one that matches the ChampSim replacement study on one trace (reseeded $\hat{\Delta}$=0.005 vs fixed 0.23). This is a *synthetic* isolation of the condition, not a real predictor: whether real PC-indexed rewards are reseed-identifiable at window grain remains untested (Table 1).

11.8 Transfer-function sensitivity

A reviewer's natural objection is that the headline percentages are artifacts of the chosen controller-collapse curve $\mu_C(u)$ and IPC (instructions-per-cycle) map. We re-run the floor (P1) and both P4 headlines across a family (Figure 15): the collapse curve as clip and as a logistic at two slopes; the IPC map steeper and concave; a *joint* $\mu_C \times$IPC change; and—the load-bearing axis—the fallback floor q_0 swept over $[0.4, 0.6]$. **The qualitative guarantees are invariant**: across every cell the steady floor violation stays $\leq 0.9\%$ with a genuine drop detected, mimicry TPR is 1.0 for competence vs 0 for the input-OOD monitor, and the secrecy ablation keeps its TPR-vs-f shape; only the exact percentages move (clean tax 0.16–0.51% across maps, on a reduced 20-episode budget per cell). Detection *power* is the one quantity that is not constant. For *any* genuine off-policy episode ($\Delta_W < \tau$) the one-sided CUSUM (a sequential change detector) accumulates a drift $K - \Delta_W \geq \gamma/2 = 0.05$, which at the boundary is $1.6\,\sigma_\Delta$ on the background/TRUSTED pool (n_{bg}=8, σ_Δ=0.031) and $3.2\,\sigma_\Delta$ only at full duty (n_L=32, σ_Δ=0.016) after Tier-A escalation; per-domain audits see less. Every tested off-policy gap gates within the 40-window horizon, but the quantity that actually varies

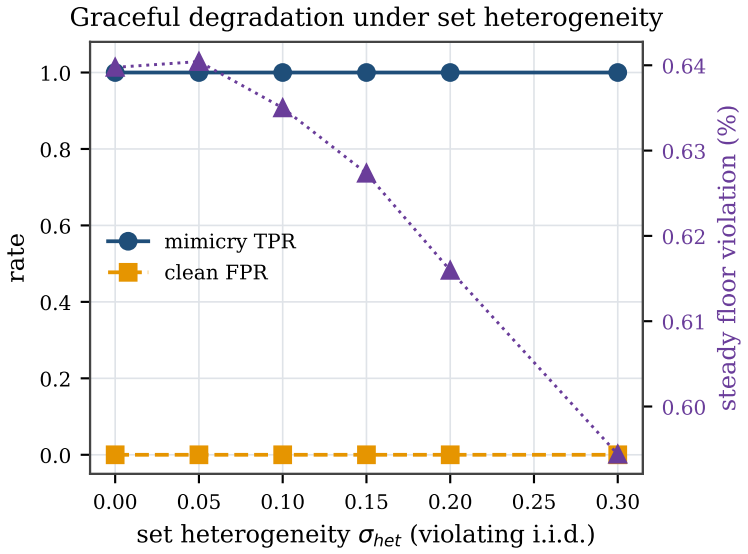


Figure 13: Graceful degradation when the i.i.d.-set assumption is violated: under set heterogeneity σ_{het} up to 0.30, mimicry TPR stays 1.0 and the floor holds; detection latency drifts from 2.0 to 2.7 windows. Temporal dependence is not varied.

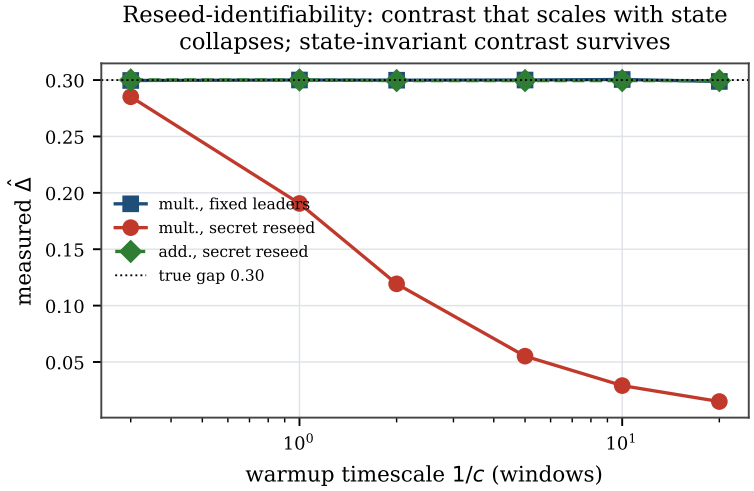


Figure 14: Synthetic warm-up model. With a multiplicative warm-up contrast, secret reseeding returns almost every sampled slice to the cold state and attenuates $\hat{\Delta}$; fixed leaders recover the 0.30 gap. An equally stateful additive warm-up with state-invariant contrast remains identifiable. This isolates a mechanism; it is not evidence about real predictors.

is the detection *latency* $D \approx H/(K - \Delta_W)$, up to $H/(K - \tau) \approx 16$ windows at the boundary, so the *fixed-deadline* TPR falls as $\Delta_W \to \tau$. We measure this directly (Figure 16): sweeping the true gap across the off-policy range, the latency rises from 2 to 13.9 windows tracking $H/(K - \Delta_W)$, and the by-10-window TPR drops from 1.0 at a deep drop to 0 at Δ_W=0.04 (recovering to 1.0 by 20 windows). Latency, not deadline-free TPR, is the supported result. *Scope:* this establishes robustness to the *shape* of the collapse curve, the IPC map, and the floor location; the Tier-A feature/confidence maps are not swept (the headline competence-beats-input result is a Tier-B property).

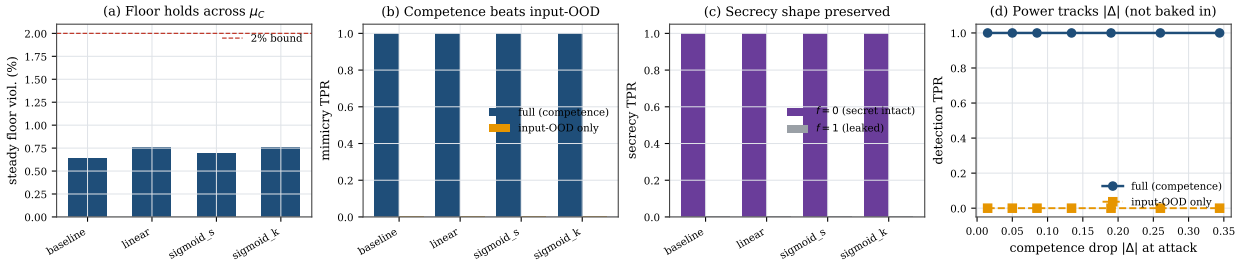


Figure 15: Transfer-function sensitivity. Across the tested collapse-curve shapes (a), the competence-beats-input mimicry result (b) and the secrecy-ablation shape (c) are stable; (d) the competence detector gates on every tested off-policy gap (within-episode TPR=1.0 vs. input-OOD 0), because the CUSUM drift $K - \Delta_W \geq 1.6\,\sigma_\Delta$ (background) throughout; what grows toward the τ boundary is the detection *latency* $D \approx H/(K - \Delta_W)$, so the fixed-deadline TPR falls (Figure 16).

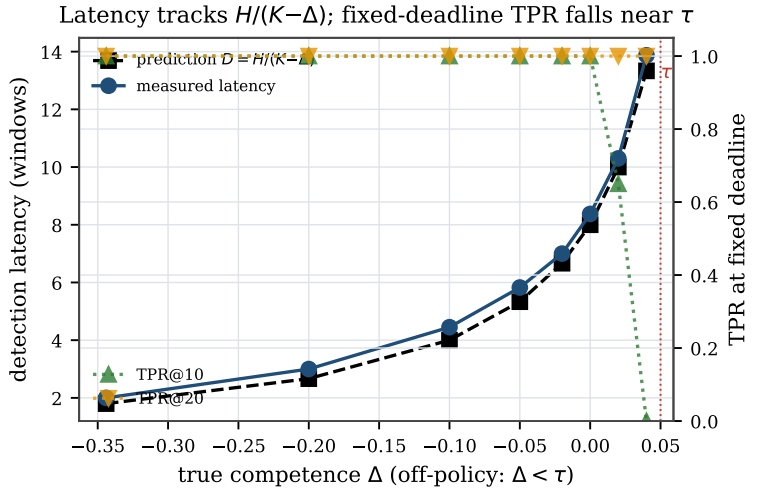
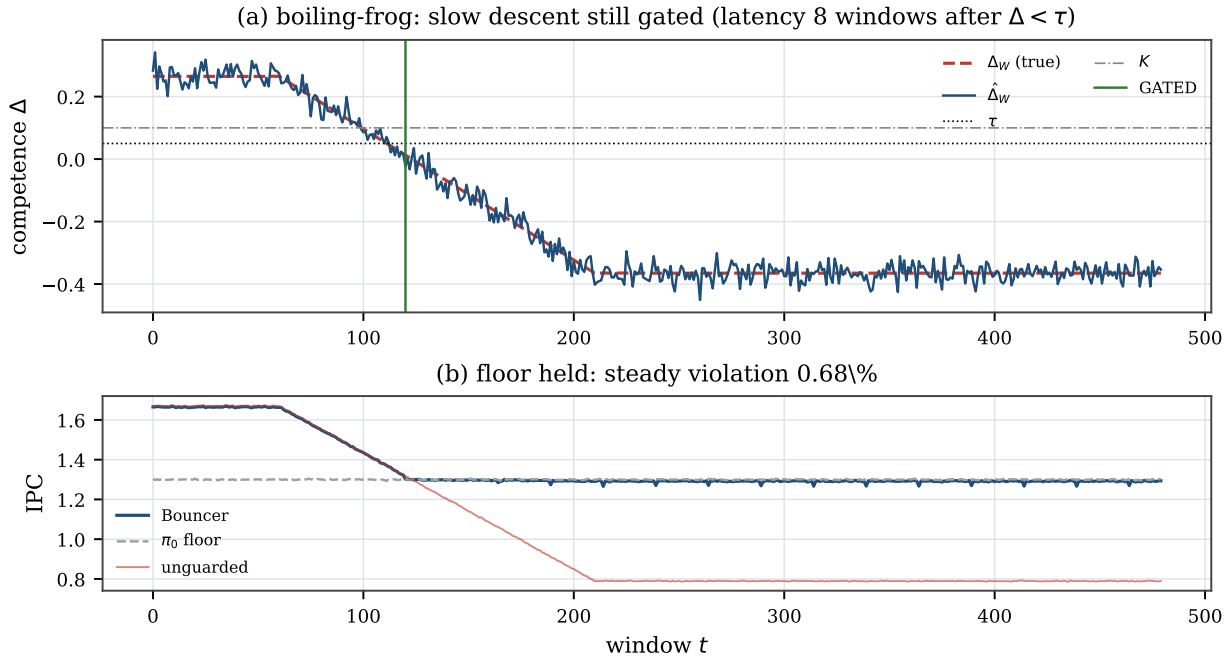


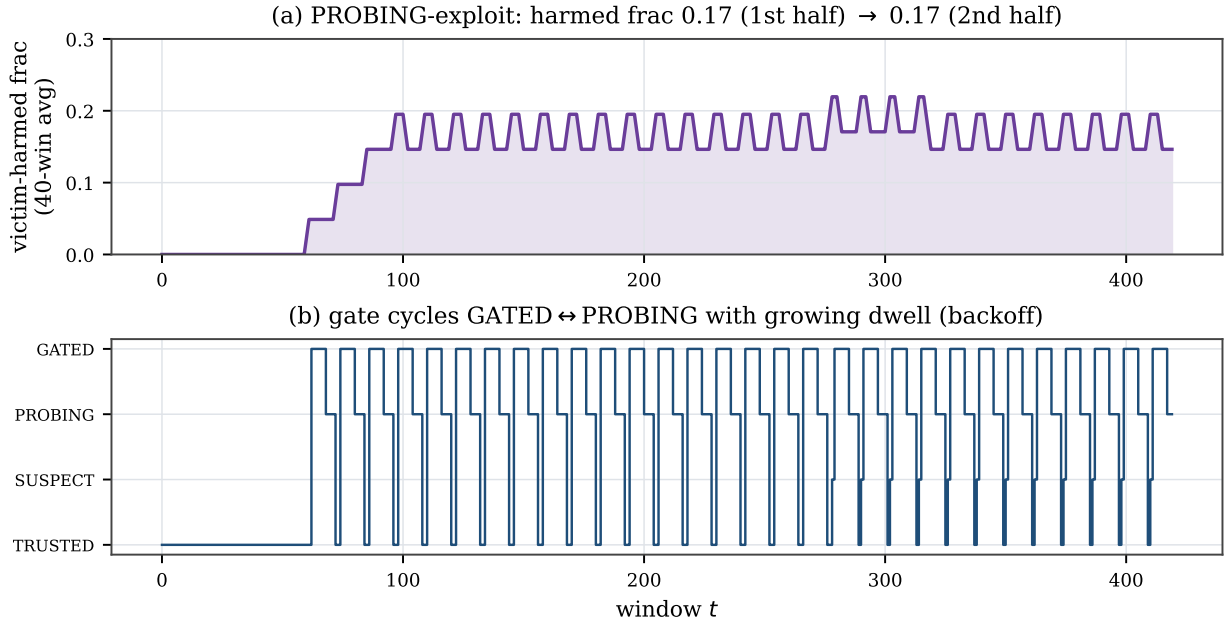
Figure 16: Detection latency vs. competence gap (positive form of Figure 15(d)). Every tested gap gates within 40 windows (full-duty drift 3.8–$28\,\sigma_\Delta$; 1.9–$14\,\sigma_\Delta$ background), but detection latency rises from 2 to 13.9 windows toward the τ boundary, tracking the deterministic-drift limit $H/(K - \Delta_W)$, so the fixed-deadline TPR falls (TPR by 10 windows drops to 0 at Δ_W=0.04, recovering to 1.0 by 20).

11.9 Adaptive timing adversaries

Two timing-based evasions test threat-model completeness (Figure 17). **Boiling-frog** (Figure 17(a)): a very slow, stealthy degradation tuned to keep the per-window change tiny. It cannot evade—a one-sided CUSUM (a running-sum change detector) integrates the *level* $(K - \hat{\Delta})$, not the slope, so once $\hat{\Delta} < K$ it accumulates regardless of how slowly it arrived. Bouncer gates 8 windows after the true $\Delta{<}\tau$ crossing (vs. 2 for an abrupt attack); the slow descent only *delays* detection, and the steady floor still holds (0.68%). **PROBING-exploit** (Figure 17(b)): a closed-loop attacker that watches the audit's decision (its gate) and backs off during the GATED/PROBING states to pass the audit, resuming when the TRUSTED state returns—the "wait out the cooldown" strategy. Because re-trust is *measured* (PROBING audits $\hat{\Delta}$) and the dwell backs off exponentially, the victim is harmed in 16.9% of windows and the attacker spends 83.1% of the horizon backed off; there is no fixed cooldown to wait out. These are finite-horizon measurements of this constructed strategy, not a general bound.



(a)



(b)

Figure 17: Adaptive timing adversaries. (a) boiling-frog is still gated (a CUSUM integrates level, not slope); (b) the PROBING-exploit attacker is bounded to a small harmed-window fraction (16.9%) by measured re-trust and dwell backoff.

11.10 Theory validation

Figure 18(a) checks the CUSUM constants on an i.i.d. Gaussian signal. Across the threshold sweep, empirical average run length and detection delay match Siegmund's approximation eq. (6): the geometric-mean

empirical/theory ratio is 0.998, and at $H = 5\sigma$ the false-alarm run length is 927 empirically versus 938 analytically. This is a formula sanity check. The deployed synthetic operating point is in a stronger-drift regime, where $H/(K - \Delta)$ is a useful mean-delay approximation.

Figure 18(b) separates the proved loose form from a descriptive tracker. For six short attack episodes, measured worst positive regret is 8.72 IPC·windows; the two-term expression is 15.14, the realized-gap/occupancy tracker is 9.48, and the loose three-term plug-in is 47.42. With three episodes of length 960, audit exposure makes the two-term expression fail: measured regret is 27.21 versus 7.57, while the tracker is 27.58 and the loose plug-in is 265.87.

The persistent-drop regression is deliberately more adversarial. Across three seeds, measured positive regret is 12.34–12.39; the descriptive tracker is 12.29 and therefore slightly *under-predicts*. The obsolete two-term expression is 2.52, while the loose three-term plug-in is 123.6. We do not call the tracker a bound: it combines realized occupancy and gap with the approximate mean delay $H/(K - \Delta)$. The Lemma proves only the loose expectation form under A2–A6. Its numeric instantiation remains conditional on using the measured/assumed episode-level D; only the exposure term has the separate high-probability envelope in eq. (5).

11.11 Traffic-weighted exposure

The audit-exposure term is traffic-weighted, and A6—a *secret uniform* sampler—is what bounds it in expectation. The released routing experiment establishes three points. *One-window concentration.* Put all of a window's traffic on one hot set: the ordinary event that the sampler tags it Leader-L (probability ϕ_G) makes the entire window run C, so the realized one-window regret is $r_{\max}$—a 15.6× violation of the naive per-window $\phi_P r_{\max}$ allowance. This is why the bound must be an *expectation*, not a per-window worst case. *Why uniform sampling matters.* If PROBING audited a *fixed* (sorted-index) prefix of the followers, a low-index hot set would be exposed with probability $1 - n_L/n_{sets} = 0.984$, not ϕ_P—a 15.4× violation of the *expectation* that no secrecy can fix. The released audit subset is instead a uniform secret subset (`set_dueling.is_audited`), and a hot set's PROBING exposure is 0.062 at low index and 0.064 at high index, both $\approx \phi_P$=0.064 and index-independent (versus 0.985/0.017 for the sorted prefix). *Horizon-level concentration.* With the uniform sampler the expected per-window exposure is $\leq \phi_P r_{\max}$ for *any* traffic (measured per-window mean 0.064 under worst-case concentration), and over independently reseeded windows the exposure total stays under the Hoeffding envelope $\phi_P T_{att} r_{\max} + r_{\max}\sqrt{(T_{att}/2)\ln(1/\delta_s)}$ (empirical coverage 1.00 at δ_s=0.01 for every $T_{att} \leq 480$).

11.12 Fully-open windows per episode: what D actually is

A2's D counts *all* fully-open (C-everywhere) windows in a drop episode—the initial detection delay plus any *false re-trust* excursions, where PROBING re-opens the gate after $T_{reprobe}$ audit estimates above $\tau + \Delta_{hys}$. We measure D *through the real controller* (`exp_retrust.py` drives the actual Tier-B CUSUM and FSM, so re-gating pays the true CUSUM delay, not a one-window surrogate—an error a reviewer caught in an earlier draft). *With the reseeded, near-i.i.d. audit* a sustained true drop yields ≈0 false re-trusts across drop depths (the audit reads $\Delta < \tau$ and rarely strings together $T_{reprobe}$ crossings), so the fully-open fraction is just the initial-detection/SUSPECT transient—0.025 (deep) to 0.073 (near τ), and $D \approx$ the initial delay. This is *not* horizon-independent in general: a temporally-*correlated* audit produces runs of crossings, so false re-trust occurs and the fully-open fraction climbs with the correlation ρ—0.008 at ρ=0 to 0.225 at ρ=0.95, again through the real controller. *Noise disclosure:* that ρ=0.95 headline injects innovation noise 0.10, which is 3.2–6.4× the harness estimator noise (σ_Δ=0.016 full duty, 0.031 background); a noise sweep at ρ=0.95 shows the fully-open fraction grows with amplitude and stays small at harness-matched amplitudes, so the 0.225 is a property of this *constructed* correlated/high-variance counterexample, not of the harness. Temporal independence is an assumption *distinct* from reseed-identifiability's signed-mean condition; we do not equate them. We therefore state D as a *measured assumption* (A2) for the reseeded regime and name the correlated-audit inflation as a *limitation*, not a bound—and every Lemma-1 numeric floor reported in this evaluation uses $D \approx$ the initial delay. Those numerals are therefore *conditional plug-in illustrations* of

eq. (4)—valid where re-trust ≈ 0 as measured here, with $H/(K-\Delta)$ a mean estimate rather than an upper bound—not standalone worst-case bounds.

11.13 Realized evasion is selection-biased; sustained evasion is not

Proposition 1's expectation and sustained-evasion forms are separated by a selection effect. *Counterexample.* Two exchangeable sets with rewards $\{1, 0\}$ randomly permuted, one secretly Leader-L, fallback 0.2, τ=0.5, β=0: unconditionally $\mathbb{E}[r_L]$=$\mathbb{E}[r_{\text{foll}}]$=0.5 (the identity holds), yet conditioned on a single window's realized evasion the follower always drew the 0— $\mathbb{E}[r_{\text{foll}} \mid \hat{\Delta}_R \geq \tau]$=0.000 with P(evade)=0.50, far below the putative 0.7 floor. Conditioning on evasion *selects* the noise, so the expectation statement does not imply a single-window realized floor. Over W independently reseeded windows, the probability that this construction's W-window average $\hat{\Delta}_R$ stays $\geq \tau$ decays under the Hoeffding envelope $e^{-W\epsilon^2/(2r_{\max}^2)}$ (range $2r_{\max}$; a W=100 Rademacher regression rejects the stronger, incorrect range-$r_{\max}$ exponent by 84.8×) (0.50 → 0.04 measured from W=1 → 16 vs. envelope 0.88 → 0.14), which is Proposition 1(ii). The same regression exercises the implemented CUSUM: a prefix-average-below-τ stream need not alarm within $2H/\gamma$, but it verifies the no-alarm⇒block-mean containment both on random degrading streams and *exhaustively* (every path of a three-point reward support from three starting statistics, 2,068 no-alarm paths, including an exact C=H boundary path, which correctly does not alarm under the strict-> semantics), and computes an *exact* (dynamic-program) no-alarm probability for an in-support two-point degrading stream (0.0038, matched by Monte Carlo on the detector and below the corrected envelope 0.726). *Operating-point check.* At the pinned operating point (n_L=32 sets, m=64 decisions, independent per-set rewards) the single-window follower selection bias is ≈0 (0.004); the catastrophe requires cross-set anticorrelation *plus* a tiny pool, and the CUSUM integrates over windows regardless.

11.14 Real-systems integration: the set-locality boundary

The auditor compiles and runs in real ChampSim as *both* a cache-replacement module (set-local by Definition 3—structurally the clean case, but stateful; see below) and an L1D prefetcher module (*not* set-local). ChampSim is the standard trace-driven CPU cache simulator. The two together test the set-locality *boundary* on real program traces in ChampSim: the prefetcher case exhibits the de-localization boundary in detail, and the replacement case shows the clean side is set-local yet stateful—a second boundary (the secret reseed confounds a path-dependent reward), rather than a real-hardware clean-case validation. We present the prefetcher case first because it is where the boundary *bites*—it is the cautionary result, not the headline.

Prefetcher (the cautionary, non-set-local case). We built a real **ChampSim L1D prefetcher module**—the same set-dueling estimator, CUSUM, and gate FSM C++ as Section 12—with the "learned controller" a per-IP stride prefetcher, the fallback prefetch-*off*, the reward the per-set cache-hit rate, and the attack a cache-polluting corruption (the dueling "sets" here are virtual block-address buckets, blk mod 2048, not physical L1D sets). It compiles and runs on SPEC CPU2017. The confirmer, CUSUM, reseed, and FSM execute off the cache-access path at window granularity. A 2-bit pool-tag lookup and output multiplexer remain per access; their RTL timing is not measured (Section 12). The real $\hat{\Delta}$ is far noisier than the synthetic one, so we ran at a looser operating point (only $m \approx 19$ accesses/bucket/window land, so $\hat{\Delta}$ swings in $[-0.8, +0.7]$) (τ=0, H=0.25) than the synthetic (τ=0.05, H=0.8, m=64); we disclose this rather than claim the synthetic R^2=0.997 estimator fidelity transfers (it does not—that is a controlled-harness result).

Across a three-trace suite (Table 3, Figure 20) the candid lesson is: the auditor is only as good as its competence reward. The *floor cap is reliable*—on every trace Bouncer ends at the prefetch-off floor, so the corruption attack's worst case is bounded: on streaming 619.lbm a corrupted prefetcher costs the unguarded config a large IPC hit (7.9%, where IPC is instructions per cycle, a throughput measure) while Bouncer (already at the floor) is unharmed; on compute-bound 600.perlbench the attack is nearly harmless (0.3%) and so is the cap. But the per-set cache-hit reward *proxy* mis-estimates prefetch competence, so the clean tax ranges 1.7–11%: on memory-bound 654.roms stride prefetching genuinely helps IPC (1.14 vs the 1.01 off floor), yet the hit-rate signal misses that benefit (it comes from latency/MLP, not raw L1D hit count), so Bouncer over-gates. Two further honesty points: the gate is *competence*-driven, not attack-label-driven—on lbm it engaged before the attack onset, so this is a floor cap, *not* attack-specific detection; and

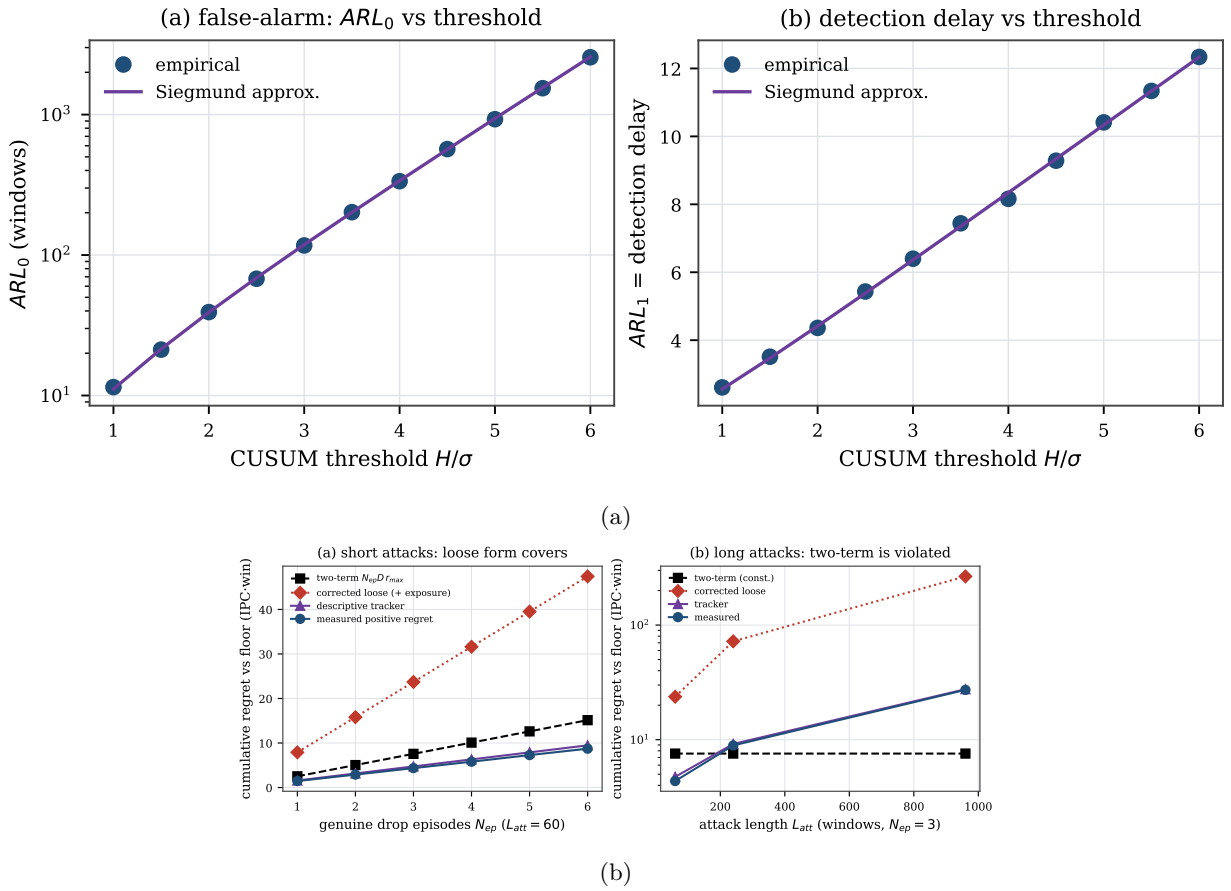


Figure 18: Theory. (a) ARL matches Siegmund; (b) measured regret versus the two-term expression, loose three-term plug-in, and descriptive tracker.

Table 3: ChampSim L1D suite (9M-insn SPEC). The floor cap bounds the attack on every trace; the cache-hit reward proxy sets the clean tax.

SPEC trace	prefetch benefit	attack loss (unguard)	Bouncer clean tax
600.perlbench (compute)	+1.7%	0.3%	1.7%
619.lbm (streaming)	≈0 (hit-rate)	7.9%	2.3%
654.roms (mem-bound)	+12.7%	5.2%	11.2%

the TRUSTED→GATED transition *dynamics* are shown where competence is controllable (the synthetic Figure 7), since no available naive-prefetcher reward exhibits a clean pre-attack TRUSTED state here.

It is tempting to blame the hit-rate proxy and propose the controller's *own* reward as the fix—but we tested exactly that and it is *insufficient*. Holding $(\tau, H, \text{window}, \text{partition}, \text{FSM})$ fixed and swapping *only* the reward, from per-set cache-hit to the prefetcher's own accuracy×timeliness, moves the roms clean tax only 11.2% → 9.7% and still loses 5% to the attacked-but-ungated prefetcher (single trace, single seed; `hardening/experiments/N1_reward_swap/`). The reason is a **fundamental tension in what set-dueling can measure**: the cache-hit reward is *set-local* (attributable to the dueling set) but a poor utility proxy, whereas the own-accuracy reward is a faithful proxy but *not set-local*—a prefetch triggered on a Leader-L set targets a *different* address that lands on a *different* set, so the dueling mis-credits it and $\hat{\Delta}$ collapses to ≈0. The dominant lever is therefore the observation window / signal SNR, not reward fidelity: lengthening the epoch (window 40k→200k) removes the over-gating even with the crude proxy. This is a genuine limitation of set-dueling *for prefetchers*, whose benefit is inherently de-localized; *cache replacement*—where the decision

and its hit/miss outcome share a set—is set-local by construction (the paper's "home-turf" condition): the structurally clean case in theory, though Section 11.14 shows a real cache's stateful reward adds a second, reseed-identifiability requirement before that cleanness is measurable. The milestone this real-simulator boundary study sets up is accordingly sharper: a long-epoch, *set-local* competence signal (latency-weighted, replacement-style), not merely "the controller's own reward."

Replacement (set-local but *stateful*—a sharper boundary). The same auditor C++ compiles and runs as a real ChampSim *LLC replacement* module: DRRIP-style dueling on *physical* cache sets (learned C = an aggressive LIP-style insertion, fallback π_0 = SRRIP-HP) scored by the demand-hit reward. Three real findings (Figure 19, 623.xalancbmk), each of which *sharpens* rather than confirms the easy "home-turf" story. *(i) The prior "$\hat{\Delta}\approx 0$" was a software artifact, not sampling.* A ChampSim replacement module's per-access hook fires only if the module also provides a fill hook (`has_cache_fill`); without it the module sees *hits only*, the demand-hit reward collapses to 1, and $\hat{\Delta}$ is *identically* 0. A cache fill is the insertion of a line on a miss, so without that hook the module never sees misses. Adding the fill hook makes $\hat{\Delta}$ a real signal—so the earlier null was a missing callback, not a horizon limit. *(ii) The whole-cache competence gap is real and large:* forcing every set to the learned policy gives a far higher LLC hit rate than the fallback (33.6% vs 4.8%; LLC is the last-level cache). *(iii) Yet the secret per-epoch reseed—the very mechanism that buys mimicry resistance—**confounds** the per-window estimate for this controller.* With reseeding, $\bar{r}_L \approx \bar{r}_F$ and $\hat{\Delta}$ sits at noise (clean mean 0.005) despite that 29-point whole-cache gap; with *fixed* leaders (reseed off, DRRIP's own configuration) the *same* estimator resolves it (clean mean $\hat{\Delta} = 0.23$). The reason is that a replacement policy's reward is *path-dependent*: a set must run one policy *consistently* to accumulate the policy-specific cache state DRRIP's PSEL counter integrates, and reseeding each epoch erases it. **Set-locality is thus necessary but not sufficient: a secret, reseeded audit additionally requires a *reseed-identifiable* policy contrast** (a near-stateless reward is the clean sufficient case)—which the synthetic harness's i.i.d. per-decision reward (A1) satisfies and a real cache, whose contrast is path-dependent, does not at per-window grain. This is a genuine secrecy/measurability tension, not a tuning issue: the clean-case competence and transition dynamics consequently live in the synthetic harness (a faithful *stateless* set-local model, Figure 7), and the deployment milestone is a reseed schedule slow enough to amortize cache-state warmup—a trade-off this result makes precise.

Disclosure (single trace, one seed). The fixed-leader per-window $\hat{\Delta}$ has std 0.57, so the resolution is a *mean-level* effect: the clean mean 0.23 is about 3 standard errors over the $\sim$50 fixed-leader windows of one trace (623.xalancbmk). The gate sat GATED in most clean windows of *both* configurations (89% reseeded, 80% fixed-leader), and the two committed logs follow different protocols (82 windows including the attack for reseeded; 56 clean-only windows for fixed-leader). A multi-trace replication (Section 14) is future work.

12 Overhead budget

Table 4 is an illustrative storage count, not a synthesized implementation. It assigns 16 bits to each coefficient or reference statistic and matches the released dimensions: a dense 16×8 projection plus eight means and standard deviations for S_{in}; 18 feature/action/bias coefficients for S_{res}; four two-sided CUSUMs; and 64 set-dueling counters. The total is 404 bytes, or 0.154% of a 256 KB SRAM.

The Python model uses floating point, so the numerical fidelity of this quantization is untested. Dynamic energy, RTL area, and timing are also unmeasured. The estimator, CUSUM, and FSM are architecturally off the access path, but the per-access 2-bit pool-tag lookup and output multiplexer remain. Consequently we claim an off-path organization, not a measured zero-latency result.

13 Related work

Learned microarchitectural controllers. A growing body of work replaces hand-tuned heuristics with online or offline-trained predictors across every major structure Bouncer targets. RL memory schedulers Ipek et al. (2008) and RL prefetchers Bera et al. (2021) learn performance-coupled policies online; learning-based cache replacement imitates Belady's optimum with PC-indexed predictors Jain & Lin (2016), distilled

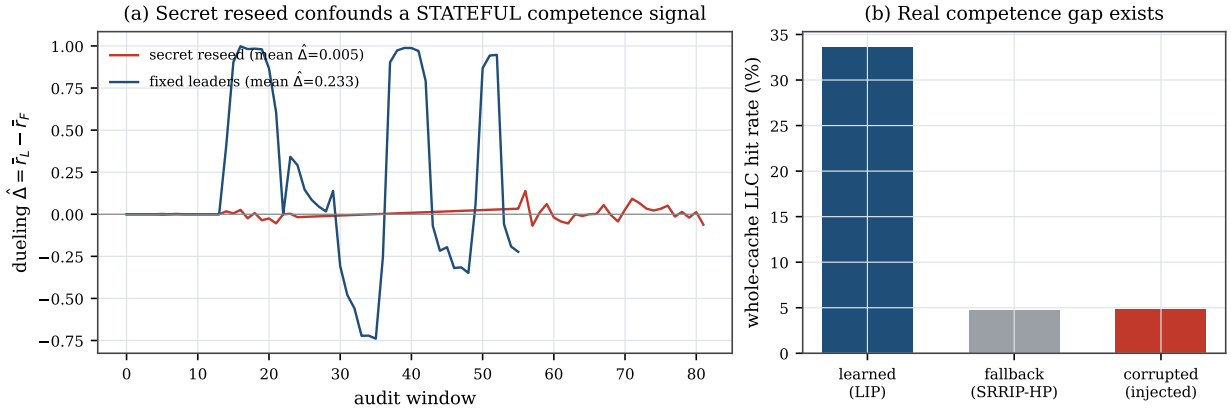


Figure 19: Real ChampSim *replacement* (623.xalancbmk). (a) The secret per-epoch reseed—load-bearing for mimicry resistance—*confounds* the per-window competence estimate for a *stateful* reward: with reseeding $\hat{\Delta}$ sits at noise; with fixed leaders the same estimator resolves the gap. (b) The whole-cache gap is real (learned vs fallback); the `cache_fill` fix makes $\hat{\Delta}$ nonzero (it was identically 0). Set-locality is necessary but not sufficient—a secret, reseeded audit also needs a reseed-identifiable contrast. This is a boundary result; the clean-case dynamics remain in the synthetic harness.

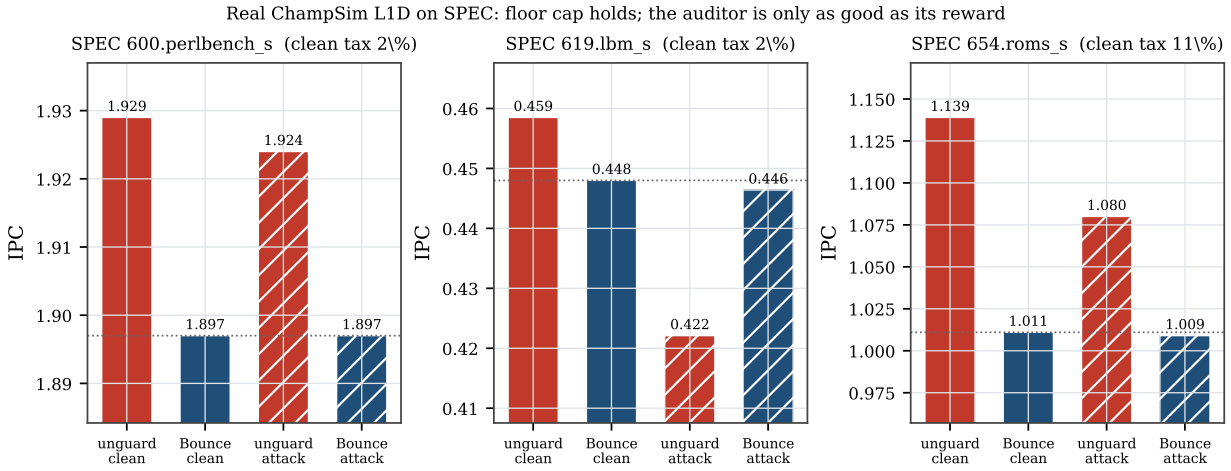


Figure 20: Bouncer as a real ChampSim L1D prefetcher across a SPEC suite. On every trace Bouncer ends at the prefetch-off floor (the cap), bounding the corruption attack's worst case (lbm: 7.9% unguarded loss cannot reach the gated victim). The per-set cache-hit reward proxy sets the clean tax (1.7–11%): roms shows the failure mode—a genuinely-helpful prefetcher under-credited and over-gated. Swapping to the controller's own reward does *not* fix this (only 11.2%→9.7%); a prefetcher's benefit is not set-local, so set-dueling needs a longer epoch or a set-local signal (replacement is the *set-local* case—though a real cache's stateful reward is a further boundary; see Section 11.14).

neural models Shi et al. (2019), or signature-based hit predictors Wu et al. (2011); and neural and geometric-history branch predictors Jiménez & Lin (2001); Seznec & Michaud (2006) have become the dominant adaptive prediction substrate. These designs share a failure surface: their advantage over a simple baseline (LRU/RRIP, a stride prefetcher, a bimodal predictor) is workload- and phase-dependent and can silently invert under distribution shift. Bouncer is agnostic to which of these controllers it wraps *among the set-local, reseed-identifiable ones it can faithfully audit*; rather than proposing a new policy, it provides, for that

Table 4: Illustrative 16-bit storage budget; not synthesized.

Component	Storage	Per-decision work
Set-dueling counters	64 B (reuse ATD)	reuse existing
S_{in} dense projection + mean/std	288 B	128 MACs on $1/k$
S_{res} forward model g	36 B	18 MACs on $1/k$
CUSUM detectors ($\times 4$)	16 B	2 add/cmp each
Total	**404 B**	estimator/gate off path

class, the missing runtime guarantee that the controller costs, in expectation, no more than a small bounded amount over its own known-safe fallback.

Set dueling and adaptive policy selection. Set dueling Qureshi et al. (2007) dedicates a handful of sampled sets to each of two competing policies and lets a single saturating counter steer the followers, a mechanism extended to re-reference prediction Jaleel et al. (2010), shared-cache thread-aware insertion Jaleel et al. (2008), and evicted-address filtering Seshadri et al. (2012), and rooted in the sampling-monitor methodology of utility-based partitioning Qureshi & Patt (2006) and sampling dead-block prediction Khan et al. (2010); a recent retrospective documents its industrial adoption and limits Qureshi et al. (2023). Bouncer borrows the dedicated-sets-plus-counter substrate but repurposes it from policy *selection* to policy *trust*: where DRRIP duels two heuristics to pick the empirical winner, Bouncer duels the learned controller against its fallback to *certify* a competence advantage and, crucially, keeps the run-C/run-fallback assignment *secret*. Classic set dueling assumes cooperative policies and is content to follow whichever wins; Bouncer must instead defend against a controller that may be steered or drifting, which is why its dueling sets feed a change detector and a trust gate rather than a preference counter.

Safe RL, runtime assurance, and Simplex. The closest architectural ancestor is Simplex Sha (2001): a simple, formally verified safety controller bounds an unverifiable high-performance one via a decision module. Shielding Alshiekh et al. (2018) synthesizes a correct-by-construction shield from a temporal-logic specification, and constrained- and Lyapunov-based safe RL Altman (1999); Achiam et al. (2017); Chow et al. (2018); Berkenkamp et al. (2017) keep policies inside a certified-safe set, with safe policy improvement Thomas et al. (2015); Laroche et al. (2019) reverting to the data-collecting baseline wherever improvement is not statistically supported (surveyed in García & Fernández (2015)). Bouncer inherits the "use simplicity to control complexity" posture and the bootstrap-to-baseline reflex, but departs in what it must prove. Simplex and shielding require a *verified model of the safety envelope*; constrained RL requires a known cost function. Bouncer needs neither: microarchitectural "safety" is not a state-space invariant but a relative performance floor, so it certifies only that the learned controller's *realized reward* beats the fallback's on randomized dueling sets, with a bounded-regret safety floor (our Lemma) that holds without any reachability proof or verified plant model. The closest relaxation of Simplex's model requirement, Black-Box Simplex Mehmood et al. (2022), still forward-simulates a reachability/admissibility check and defines safety as a state-space invariant; Bouncer instead defines it as a measured relative-competence floor and needs no plant model at all. Concurrent runtime-assurance work certifies recovery *deadlines* for adapting controllers via conformal prediction Shojaei (2026); Bouncer is complementary, certifying a competence floor rather than a recovery time, and adds the secrecy-based mimicry resistance that an observable certificate lacks.

Change detection and OOD monitoring. Tier-B runs a one-sided CUSUM Page (1954), using quickest-detection theory Lorden (1971); Moustakides (1986) and sequential threshold sizing Siegmund (1985); Tartakovsky et al. (2014). Tier-A instead draws on input monitoring: confidence baselines Hendrycks & Gimpel (2017), conformal validity Vovk et al. (2005); Lei et al. (2018), concept-drift detection Gama et al. (2014), and representation-level shift tests Rabanser et al. (2019). Such monitors measure whether inputs move; Bouncer's duel measures whether realized controller reward falls relative to a fallback.

Related label-free deterioration tests include the Suitability Filter Pouget et al. (2025), sequential harmful-shift detection Amoukou et al. (2024), and D3M Nguyen et al. (2025). Idealized D3M provides the exact false-alarm guarantee; its practical variant approximately inherits it. Bouncer's distinction is a secret, reseeded, hardware-oriented competence sample with an expected-regret gate. The illustrative storage count

is 404 bytes, but the proposed quantization and RTL timing are unvalidated. Its attribution also requires representative sampling, set-locality, and reseed-identifiability.

Microarchitectural security and co-tenant attacks. The controllers Bouncer audits are also attack surfaces. Contention and timing channels on caches Osvik et al. (2006); Yarom & Falkner (2014); Liu et al. (2015); Gruss et al. (2016b) and occupancy channels Shusterman et al. (2019) leak across cores and VMs in shared clouds Ristenpart et al. (2009), while branch predictors Evtyushkin et al. (2016; 2018) and prefetchers Gruss et al. (2016a); Guo et al. (2022) expose secret adaptive state that an attacker can read out or steer. A learned controller widens this surface: its policy can be poisoned into amplifying leakage or into co-tenant denial of service. Prior defenses harden specific channels; Bouncer is orthogonal and complementary, bounding the *aggregate* damage any steered controller can do by capping its cost at the vetted fallback. Its mimicry resistance (our Proposition) follows precisely from the secrecy of the set assignment, so an attacker who can observe and game stealthy counter-based detectors Gruss et al. (2016b) still cannot tell which accesses are being scored.

ML-for-systems robustness. Deployed learned components are brittle in exactly the regimes Bouncer guards. Oracle-imitation cache controllers lose accuracy off-distribution Liu et al. (2020), learned indexes degrade on irregular data Marcus et al. (2020), and the Park benchmark documents why RL-for-systems policies behave unlike RL-for-games under noisy, non-stationary workloads Mao et al. (2019); this is the hidden, system-level risk catalogued in Sculley et al. (2015). Production systems respond with bespoke guardrails: SLO-clamped colocation Lo et al. (2015), OOM-bounded autoscaling Rzadca et al. (2020), and fault-triggered fallback in learned software Carbune et al. (2023). Bouncer generalizes these one-off, fault- or threshold-triggered safeguards into a single hardware-cheap mechanism with a *continuous* competence audit and a provable performance floor, applicable to any *set-local, reseed-identifiable* learned microarchitectural controller rather than one tuned guardrail per deployment.

14 Discussion and limitations

Counterfactual cost. Controllers without clean set-sampling (the memory scheduler) force a model-based $\hat{\Delta}$, which weakens Proposition 1; we scope sampling-friendly controllers first. **Redistributive mimicry and the coverage hole.** A global $\hat{\Delta}$ cannot see an attack that preserves the global mean; per-domain dueling raises detection but does *not* fully close it—a slow-bleed adversary below the resolution floor $\delta_R^{\min}(B)$ evades indefinitely (Proposition 1, the coverage hole of Figure 5). We name it rather than hide it. **Ground-truth labeling.** "Off-policy" is operationalized as a sustained competence drop beyond a fixed margin, against which we measure latency. **Overhead vs. sensitivity.** The auditor dies if the Tier-B duty cycle is too high; the minimal-overhead operating point—the knee of Figure 9(a)—is reported as the result, not a footnote. **From simulation to silicon.** The controlled quantitative claims are validated on a faithful harness. The real ChampSim L1D module (Section 11.14) shows the mechanism survives a cycle-accurate simulator and bounds a corrupted prefetcher to the safe floor across a 3-trace SPEC suite; a full SPEC/GAP sweep with a timeliness-aware reward, and the production controllers of Table 1, is the natural next step. **Faithful attribution needs all three conditions.** Representative within-domain sampling aligns the duel with deployed traffic; set-locality aligns each decision with its credited outcome; and reseed-identifiability preserves the policy contrast after leaders are shuffled. The per-set cache-hit reward over-gates a genuinely helpful prefetcher on roms (Section 11.14, 11% tax); swapping to the controller's *own* accuracy×timeliness reward does *not* fix it (11.2%→9.7%), because a prefetcher's benefit is intrinsically de-localized (it fetches a different set than it was triggered on) and cannot be dualized to any per-set reward. The structurally clean case is a controller whose decision and outcome share a set—*replacement*—a property of reward *geometry* that sharply scopes where set-dueling competence auditing is faithful.

15 Conclusion

Bouncer turns "is the learned controller still earning its keep?" into one measurable scalar and bounds the cost of the answer. Its organizing result is the *set-locality* condition (Definition 3): a secret per-set competence audit can attribute reward to the policy that earned it only when a controller's reward shares a

set with its decision. That makes cache *replacement* the clean set-local case and prefetching the de-localized boundary. The real-systems study adds a second condition: set-locality is necessary but not sufficient, because a real cache is set-local yet *stateful*, so the secret reseed that buys mimicry resistance confounds its per-window measurement—a tension that a *reseed-identifiable* contrast resolves (Definition 4). Both controller-side conditions presuppose that the within-domain sample is representative of deployed traffic, or is explicitly traffic-weighted.

For a controller in that class, Bouncer repurposes the set-dueling substrate into a secret competence audit, gates an expensive confirmer behind cheap tripwires, and proves a three-term safety floor—detection, audit-exposure, and false-alarm—tied to the constants of the CUSUM detector. The payoff: for declared domains audited at resolution $\delta_R^{\min}(B)$ with the secret intact, Bouncer bounds the *expected* competence regret, in the controller's own reward, against its vetted fallback, under benign drift and an adaptive mimicry adversary alike, for 404 bytes in an illustrative 16-bit storage count. RTL area, energy, timing, and quantized numerical fidelity are not evaluated. The robustness is neither free nor unconditional: the competence signal is what beats an input monitor, and its resistance to a *leak-aware* attacker is purchased by the secrecy of the dueling sets—we quantify the price of losing that secrecy, and we name the sub-resolution slow-bleed adversary outside the audited domain as a real, un-closed vulnerability.

Broader Impact

Bouncer is a defensive runtime monitor: it bounds the expected competence cost of a learned microarchitectural controller against a vetted fallback, which serves a co-tenant denial-of-service defense role in shared clouds. Two dual-use considerations. First, the secrecy of the reseeded dueling assignment is what defeats a *leak-aware* attacker (competence measurement is what defeats plain input mimicry); the same primitive could gate a shared controller for censorship-style control, but the mimicry analysis (Proposition 1) also bounds what such gating can hide, since a global $\hat{\Delta}$ cannot conceal a redistributive harm from a per-domain audit. Second, we publish the sub-resolution slow-bleed coverage hole (Figure 5) rather than hide it: naming the resolution floor $\delta_R^{\min}(B) \propto 1/\sqrt{B}$ lets defenders budget leader counts against a target harm rate, which we judge net beneficial because the attack is derivable from the public concentration bound regardless. Online-weight poisoning is out of scope and stated as such (Section 3).

Reproducibility Statement

All synthetic results, figures, and this PDF regenerate deterministically via `./run_all.sh` (CPython 3.11 with the versions pinned in `requirements.txt`; seeded RNG). `scripts/check_paper_numbers.py` (`run_all` stage 9) is a *literal-presence audit*: it checks that headline numerals rendered from the released `results/*.json` appear in the source. It is a regression tripwire for numeral drift, not an exhaustive semantic audit of every claim. We check rendered strings rather than byte-identity because floating-point results shift by 1–2 ulps across numpy builds (the committed `hardening/manifests/` sha256 files are a provenance record of one such build, not a portability claim). The harness invariants (single-assignment-per-window; reseed) are checked by `experiments/test_invariants.py` (stage 0). The ChampSim figures in `run_all` are regenerated from committed JSON/CSV logs, not from a fresh simulator run. `champsim_plugin/setup_champsim.sh` builds and runs the `lbm` prefetcher case; `run_e1_replacement.sh` runs the single-trace replacement case. Neither script regenerates the full three-trace suite in one command. Lemma 1 has two standalone regressions: `experiments/exp_floor_longattack.py` (the long-attack audit-exposure envelope) and `experiments/exp_floor_traffic.py` (the traffic-weighting counterexample and the expectation/high-probability repair).

Artifact Appendix

The full artifact is released: the auditor and environment (`bouncer/`), one script per evaluation phase (`experiments/`), including the transfer-function sensitivity sweep (`exp_e3_sensitivity.py`, Section 11.8), all result JSON and figures, the real ChampSim L1D prefetcher *and* LLC replacement modules

(`champsim_plugin/bouncer_pref/`, `bouncer_repl/`), and the paper source. `./run_all.sh` regenerates every synthetic result, figure, and the PDF (machine-dependent: ∼5 minutes on a fast laptop, ∼20 minutes in a constrained sandbox; CPython 3.11, pinned numpy/scipy/matplotlib/pandas); all RNG is explicitly seeded, so results are deterministic. `champsim_plugin/setup_champsim.sh` clones and builds ChampSim, installs the Bouncer module, fetches a public SPEC CPU2017 `lbm` trace, and runs that prefetcher case; `champsim_plugin/run_e1_replacement.sh` reproduces the Section 11.14 replacement boundary study (Figure 19) on `623.xalancbmk` (C++17 toolchain + vcpkg). The standalone auditor additionally builds with `c++ -std=c++17 bouncer.cc -o bouncer_selftest`.

References

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pp. 22–31. PMLR, 2017.

Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*, pp. 2669–2678. AAAI Press, 2018.

Eitan Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

Salim I. Amoukou, Tom Bewley, Saumitra Mishra, Freddy Lecue, Daniele Magazzeni, and Manuela Veloso. Sequential harmful shift detection without labels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2412.12910.

Rahul Bera, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1121–1137, 2021. doi: 10.1145/3466752.3480114.

Felix Berkenkamp, Matteo Turchetta, Angela P. Schoellig, and Andreas Krause. Safe model-based reinforcement learning with stability guarantees. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 908–918, 2017.

Victor Carbune, Thierry Coppey, Alexander Daryin, Thomas Deselaers, Nikhil Sarda, and Jay Yagnik. Smartchoices: Augmenting software with learned implementations. *arXiv preprint arXiv:2304.13033*, 2023.

Yinlam Chow, Ofir Nachum, Edgar Duenez-Guzman, and Mohammad Ghavamzadeh. A lyapunov-based approach to safe reinforcement learning. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, pp. 8092–8101, 2018.

Dmitry Evtyushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. Jump over ASLR: Attacking branch predictors to bypass ASLR. In *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1–13. IEEE, 2016. doi: 10.1109/MICRO.2016.7783743.

Dmitry Evtyushkin, Ryan Riley, Nael B. Abu-Ghazaleh, and Dmitry Ponomarev. BranchScope: A new side-channel attack on directional branch predictor. In *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 693–707. ACM, 2018. doi: 10.1145/3173162.3173204.

João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):44:1–44:37, 2014. doi: 10.1145/2523813.

Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16:1437–1480, 2015.

Daniel Gruss, Clémentine Maurice, Anders Fogh, Moritz Lipp, and Stefan Mangard. Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 368–379. ACM, 2016a. doi: 10.1145/2976749.2978356.

Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. Flush+flush: A fast and stealthy cache attack. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, volume 9721 of *Lecture Notes in Computer Science*, pp. 279–299. Springer, 2016b. doi: 10.1007/978-3-319-40667-1_14.

Yanan Guo, Andrew Zigerelli, Youtao Zhang, and Jun Yang. Adversarial prefetch: New cross-core cache side channel attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 1458–1473. IEEE, 2022. doi: 10.1109/SP46214.2022.9833692.

Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Engin Ipek, Onur Mutlu, José F. Martínez, and Rich Caruana. Self-optimizing memory controllers: A reinforcement learning approach. In *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA)*, pp. 39–50, 2008. doi: 10.1109/ISCA.2008.21.

Akanksha Jain and Calvin Lin. Back to the future: Leveraging belady's algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, pp. 78–89, 2016. doi: 10.1109/ISCA.2016.17.

Aamer Jaleel, William Hasenplaugh, Moinuddin K. Qureshi, Julien Sebot, Simon C. Steely, and Joel Emer. Adaptive insertion policies for managing shared caches. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pp. 208–219. ACM, 2008. doi: 10.1145/1454115.1454145.

Aamer Jaleel, Kevin B. Theobald, Simon C. Steely, and Joel Emer. High performance cache replacement using re-reference interval prediction (rrip). In *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*, pp. 60–71. ACM, 2010. doi: 10.1145/1815961.1815971.

Daniel A. Jiménez and Calvin Lin. Dynamic branch prediction with perceptrons. In *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 197–206, 2001. doi: 10.1109/HPCA.2001.903263.

Samira Manabi Khan, Yingying Tian, and Daniel A. Jiménez. Sampling dead block prediction for last-level caches. In *Proceedings of the 43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 175–186. IEEE Computer Society, 2010. doi: 10.1109/MICRO.2010.24.

Romain Laroche, Paul Trichelair, and Rémi Tachet des Combes. Safe policy improvement with baseline bootstrapping. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3652–3661. PMLR, 2019.

Jing Lei, Max G'Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523):1094–1111, 2018. doi: 10.1080/01621459.2017.1307116.

Evan Zheran Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. An imitation learning approach for cache replacement. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 6237–6247, 2020.

Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. Last-level cache side-channel attacks are practical. In *2015 IEEE Symposium on Security and Privacy (SP)*, pp. 605–622. IEEE Computer Society, 2015. doi: 10.1109/SP.2015.43.

David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. Heracles: Improving resource efficiency at scale. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, pp. 450–462, 2015. doi: 10.1145/2749469.2749475.

G. Lorden. Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics*, 42 (6):1897–1908, 1971. doi: 10.1214/aoms/1177693055.

Hongzi Mao, Parimarjan Negi, Akshay Narayan, Hanrui Wang, Jiacheng Yang, Haonan Wang, Ryan Marcus, Ravichandra Addanki, Mehrdad Khani Shirkoohi, Songtao He, Vikram Nathan, Frank Cangialosi, Shaileshh Bojja Venkatakrishnan, Wei-Hung Weng, Song Han, Tim Kraska, and Mohammad Alizadeh. Park: An open platform for learning-augmented computer systems. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pp. 2490–2502, 2019.

Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. Benchmarking learned indexes. *Proceedings of the VLDB Endowment (PVLDB)*, 14(1):1–13, 2020. doi: 10.14778/3421424.3421425.

Usama Mehmood et al. The black-box simplex architecture for runtime assurance of autonomous CPS. In *NASA Formal Methods (NFM)*, 2022. arXiv:2102.12981.

George V. Moustakides. Optimal stopping times for detecting changes in distributions. *The Annals of Statistics*, 14(4):1379–1387, 1986. doi: 10.1214/aos/1176350164.

Viet Nguyen, Changjian Shui, Vijay Giri, Siddharth Arya, Amol Verma, Fahad Razak, and Rahul G. Krishnan. Reliably detecting model failures in deployment without labels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. D3M; arXiv:2506.05047.

Dag Arne Osvik, Adi Shamir, and Eran Tromer. Cache attacks and countermeasures: The case of AES. In *Topics in Cryptology – CT-RSA 2006, The Cryptographers' Track at the RSA Conference*, volume 3860 of *Lecture Notes in Computer Science*, pp. 1–20. Springer, 2006. doi: 10.1007/11605805_1.

E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954. doi: 10.1093/biomet/41.1-2.100.

Angéline Pouget, Mohammad Yaghini, Stephan Rabanser, and Nicolas Papernot. Suitability filter: A statistical framework for classifier evaluation in real-world deployment settings. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2505.22356.

Moinuddin K. Qureshi and Yale N. Patt. Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches. In *Proceedings of the 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 423–432. IEEE Computer Society, 2006. doi: 10.1109/MICRO.2006.49.

Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely, and Joel Emer. Adaptive insertion policies for high performance caching. In *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*, pp. 381–391. ACM, 2007. doi: 10.1145/1250662.1250709.

Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely, and Joel Emer. Retrospective: Adaptive insertion policies for high-performance caching. In *ISCA@50 25-Year Retrospective: 1996–2020*, 2023. Retrospective on the ISCA 2007 DIP/set-dueling paper.

Stephan Rabanser, Stephan Günnemann, and Zachary C. Lipton. Failing loudly: An empirical study of methods for detecting dataset shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

Thomas Ristenpart, Eran Tromer, Hovav Shacham, and Stefan Savage. Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds. In *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS)*, pp. 199–212. ACM, 2009. doi: 10.1145/1653662.1653687.

Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, pp. 16:1–16:16, 2020. doi: 10.1145/3342195.3387524.

D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pp. 2503–2511, 2015.

Vivek Seshadri, Onur Mutlu, Michael A. Kozuch, and Todd C. Mowry. The evicted-address filter: A unified mechanism to address both cache pollution and thrashing. In *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pp. 355–366. ACM, 2012. doi: 10.1145/2370816.2370868.

André Seznec and Pierre Michaud. A case for (partially) TAgged GEometric history length branch prediction. *Journal of Instruction-Level Parallelism (JILP)*, 8:1–23, 2006.

Lui Sha. Using simplicity to control complexity. *IEEE Software*, 18(4):20–28, 2001. doi: 10.1109/MS.2001.936213.

Zhan Shi, Xiangru Huang, Akanksha Jain, and Calvin Lin. Applying deep learning to the cache replacement problem. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 413–425, 2019. doi: 10.1145/3352460.3358319.

Alireza Shojaei. Conformal recovery-deadline certificates for runtime assurance of adapting controllers. *arXiv preprint arXiv:2606.25371*, 2026.

Anatoly Shusterman, Lachlan Kang, Yarden Haskal, Yosef Meltser, Prateek Mittal, Yossi Oren, and Yuval Yarom. Robust website fingerprinting through the cache occupancy channel. In *28th USENIX Security Symposium (USENIX Security 19)*, pp. 639–656, Santa Clara, CA, 2019. USENIX Association.

David Siegmund. *Sequential Analysis: Tests and Confidence Intervals*. Springer Series in Statistics. Springer-Verlag, New York, 1985. doi: 10.1007/978-1-4757-1862-1.

Alexander Tartakovsky, Igor Nikiforov, and Michèle Basseville. *Sequential Analysis: Hypothesis Testing and Changepoint Detection*. Monographs on Statistics and Applied Probability. Chapman & Hall/CRC, Boca Raton, FL, 2014. ISBN 978-1-4398-3820-4.

Philip S. Thomas, Georgios Theocharous, and Mohammad Ghavamzadeh. High confidence policy improvement. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pp. 2380–2388. PMLR, 2015.

Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, New York, 2005. doi: 10.1007/b106715.

Carole-Jean Wu, Aamer Jaleel, William Hasenplaugh, Margaret Martonosi, Simon C. Steely, and Joel Emer. SHiP: Signature-based hit predictor for high performance caching. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 430–441. ACM, 2011. doi: 10.1145/2155620.2155671.

Yuval Yarom and Katrina Falkner. FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In *23rd USENIX Security Symposium (USENIX Security 14)*, pp. 719–732, San Diego, CA, 2014. USENIX Association.